14 haftalık yol haritasında 2. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 02

Algoritma Analizi: İşlem Sayma, Asimptotik Gösterim ve Büyüme Hızları

RAM modeli ve temel işlem, en iyi/ortalama/en kötü durum, O–Ω–Θ, büyüme sınıfları ve uygulanabilirlik, oran testi ve log–log eğim; deneme bölmesi, Eratosthenes kalburu, hızlı üs alma ve Kadane

MODÜL I · ALGORİTMALARIN DİLİ · H1–H3

HAFTANIN BÜYÜK SORUSU

10^6 elemanda 1 saniyede çalışan bir program 10^9 elemanda ne kadar sürer ve bu soru, programın hangi dilde yazıldığından neden daha önemlidir?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 3
2. Geçen Haftadan: Doğruluktan Verimliliğe 3
3. Kavramsal Anlatım: Maliyeti Ölçmeden Kestirmek 4
3.1 RAM modeli, temel işlem ve T(n) 4
3.2 En iyi, en kötü ve ortalama durum 5
3.3 Asimptotik gösterim: O, Ω ve Θ 5
3.4 Döngülerden karmaşıklık 6
3.5 Büyüme sınıfları ve uygulanabilirlik 8
3.6 Deneysel analiz: oran testi ve log–log eğim 9
3.7 Bellek karmaşıklığı ve Kaputun Altında 10
4. Haftanın Algoritmaları: Sayı Kuramından Alt Dizilere 11
4.1 Deneme bölmesiyle asallık: √n sınırı 11
4.2 Eratosthenes kalburu: zaman–bellek ödünleşimi 14
4.3 Hızlı üs alma ve modüler üs: kare al ve çarp 17
4.4 En büyük alt dizi toplamı: $O(n^3) \rightarrow O(n^2) \rightarrow O(n)$ (Kadane) 20
5. Çözümlü Örnekler: Merdivenin Kalan Basamakları 23
6. Haftanın Deneyi: Üç Gizemli Fonksiyonu Sınıflandırmak 26
7. Hata Avcısı: Gizli Maliyetler ve Yanlış Sonuçlar 28
8. Dil Penceresi: Aynı Görünen Kod, Farklı Maliyet 30
9. Mühendis Gözüyle: Karmaşıklık Bütçesi ve Gizli Karesel Kod 31
10. Mini-OBS 2 Adımı: v2.1 — Sayaçlar ve Yinelenen Numara 31
11. Alıştırmalar ve Atölye 33
11.1 Alıştırmalar (A–G) 33
11.2 Atölye 1 · “Karmaşıklığını söyle” kart oyunu (12 kart) 37
11.3 Atölye 2 · Oran testi deneyi: üç gizemli fonksiyon 38
11.4 Atölye 3 · Kadane yarışması 39
11.5 Atölye 4 · Asal sayma: $\pi(10^7) = 664\,579$ 39
12. Kendini Değerlendirme 40
13. İleri Bakış ve Merak Kutusu 40
14. Özet, Kavram Haritası ve Terim Sözlüğü 40
15. Kaynaklar 42
Ek A. Dil Karşılıkları: C++ ve Python Listeleri 43

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

10^6 elemanda 1 saniyede çalışan bir program 10^9 elemanda ne kadar sürer ve bu soru, programın hangi dilde yazıldığından neden daha önemlidir?

H1'de bir programın **doğru** olduğunu döngü değişmeziyle gerekçelendirdik ve süresini dikkatle ölçmeyi öğrendik. Mini-OBS 2 v2.0'da bu ölçüm düzeni AP-1'den gelen yükleyicinin beklenmedik bir yüzünü gösterdi: 10^5 kayıt 17,1 saniyede yükleniyor, 10^6 kayıt için kestirim ≈ 1,6 saat; log–log eğim 2,52. Bu hafta soruyu tersinden soruyoruz: süre n ile **nasıl büyüyecek**, programı çalıştırmadan, koduna bakarak söyleyebilir miyiz? Aracımız **algoritma analizi** (algorithm analysis) ve **asimptotik gösterim** (asymptotic notation); Büyük Soru'nun yanıtı Bölüm 3.5'tedir.

> GERÇEK DÜNYADAN · **2021: GTA Online'ın altı dakikalık yükleme ekranı**
>
> Takma adı **t0st** olan bir oyuncu, Grand Theft Auto Online'ın çevrim içi modunun kendi bilgisayarında (AMD FX-8350) yaklaşık **6 dakikada** yüklendiğini gördü: ilk dakikadaki ortak kaynak yüklemesinden sonra yaklaşık dört dakika boyunca tek bir işlemci çekirdeği tam yükte çalışıyor, disk ve ağ ise neredeyse boşta kalıyordu. Süreyi iki **gizli karesel** desen yiyordu: (1) yaklaşık **10 MB**'lık bir JSON metni ayrıştırılırken çağrılan `sscanf`, her çağrıda dizginin uzunluğunu `strlen` ile baştan hesaplıyordu; (2) yaklaşık **63 000** öğenin her biri diziye eklenmeden önce dizideki bütün öğelerle karşılaştırılıyordu: yazarın hesabıyla 1 984 531 500 denetim (≈ $n^2/2$), üstelik öğeler zaten benzersizdi.
>
> Yalnızca yinelenen denetimini kaldırmak süreyi 4 dk 30 s'ye, yalnızca uzunluğu önbelleğe almak 2 dk 50 s'ye, ikisi birlikte 1 dk 50 s'ye indirdi: yazarın hesabıyla %69,4 kısalma. Yazının sonundaki güncellemeye göre düzeltme 16 Mart 2021'de oyuna eklendi (t0st, 2021). Kod "doğruydu"; sorun **büyüme hızıydı**. Mini-OBS v1.0 yükleyicisinin "mükerrer numara kuralı" da aynı ikinci desendir.

Öğrenme çıktıları

ÖÇ 2.1 RAM modeli altında temel işlemleri sayarak bir algoritmanın çalışma süresini T(n) fonksiyonu olarak ifade eder. *(3.1)*

ÖÇ 2.2 En iyi, en kötü ve ortalama durumu ayırt eder; basit algoritmalar için her birini belirler. *(3.2)*

ÖÇ 2.3 O, Ω ve Θ gösterimlerini tanımlarına uygun kullanır; fonksiyonları büyüme hızlarına göre sıralar. *(3.3, 3.5)*

ÖÇ 2.4 Döngü yapılarından (ardışık, iç içe, bağımlı, yarıya inen) karmaşıklık sınıfını çıkarır ve oran testiyle deneysel olarak doğrular. *(3.4, 3.6, Bölüm 6)*

ÖÇ 2.5 Sayı kuramı algoritmalarını (deneme bölmesi, Eratosthenes kalburu, Öklid, hızlı üs alma) zaman ve bellek karmaşıklıkları açısından karşılaştırır. *(Bölüm 4, 5.3)*

2. Geçen Haftadan: Doğruluktan Verimliliğe

H1'in üç aracı bu hafta da masada: **döngü değişmezi**, **kâhinle karşılaştırmalı test** ve **dikkatli ölçüm** (ısınma, tekrar, ortanca; süreler stderr'e). AP-1'den üç gözlem bu haftanın konusudur: H9'da özyinelemeli Fibonacci'nin çağrı sayısı üstel büyüyordu (**işlem sayısının büyümesi**); H11'de aynı matrisi satır satır ve sütun sütun dolaşmak aynı işlem sayısına karşın farklı sürelerde bitiyordu (**birim maliyetin sınırları**); H13'te `strlen`'i yazarken dizginin bütün karakterlerini taradık (**gizli maliyet**).

> **TAHMİN ET · Geri çağırma: üç soru**
>
> - **S1.** H1'deki doğrusal arama, aranan değer dizide yoksa 10^6 elemanda kaç karşılaştırma yapar? $2 \cdot 10^6$ elemanda?
> - **S2.** Bir süreyi neden tek ölçümle değil tekrarların **ortancasıyla** raporluyoruz?
> - **S3.** `-O2` ile derlenen bir döngünün süresi 0,0000 ms çıktı. İlk şüpheniz ne olmalı?
>
> Yanıtlar: S1 — 10^6 ve $2 \cdot 10^6$: girdi iki katına çıkınca iş de iki katına çıkar. S2 — tek ölçüm araya giren süreçlerden etkilenir; ortanca aykırı değerlere dayanıklıdır. S3 — derleyici sonucu kullanılmayan hesabı ölü kod sayıp silmiş olabilir (Hata Avcısı 8).

3. Kavramsal Anlatım: Maliyeti Ölçmeden Kestirmek

3.1 RAM modeli, temel işlem ve T(n)

Ölçülen süre makineye, derleyiciye, dile ve o anda çalışan başka programlara bağlıdır. İki algoritmayı bunlardan bağımsız karşılaştırmak için bir makine modeli seçeriz: **RAM modeli** (random-access machine). Komutlar sırayla yürür; aritmetik, karşılaştırma, atama, indisle dizi erişimi, fonksiyon çağrısı ve dönüşü **birer birim** maliyetlidir; sözcük boyu sınırlıdır (Cormen vd., 2022). **Çalışma süresi** T(n), n boyutlu bir girdide yürütülen birim adımların sayısıdır. Bütün adımları saymak yerine işi temsil eden **temel işlemi** (basic operation) seçip yalnızca onu sayarız: toplamada toplama, aramada karşılaştırma, asallık testinde kalan (%) işlemi. Örnek 1 aynı döngüyü iki biçimde sayar (Tablo 2.1).

Tablo 2.1
Aynı döngü, iki maliyet modeli (Örnek 1)

Adım (sözde kod)	RAM modeli	Temel işlem: toplama
toplam ← 0 ve i ← 0	2	—
i < n sınaması	n + 1	—
toplam ← toplam + a[i]	n	n
i ← i + 1	n	—
Toplam (kapalı biçim)	**T(n) = 3n + 3**	**T(n) = n**

C **H02_01_dogrusal_toplam.c · satır 19–28**

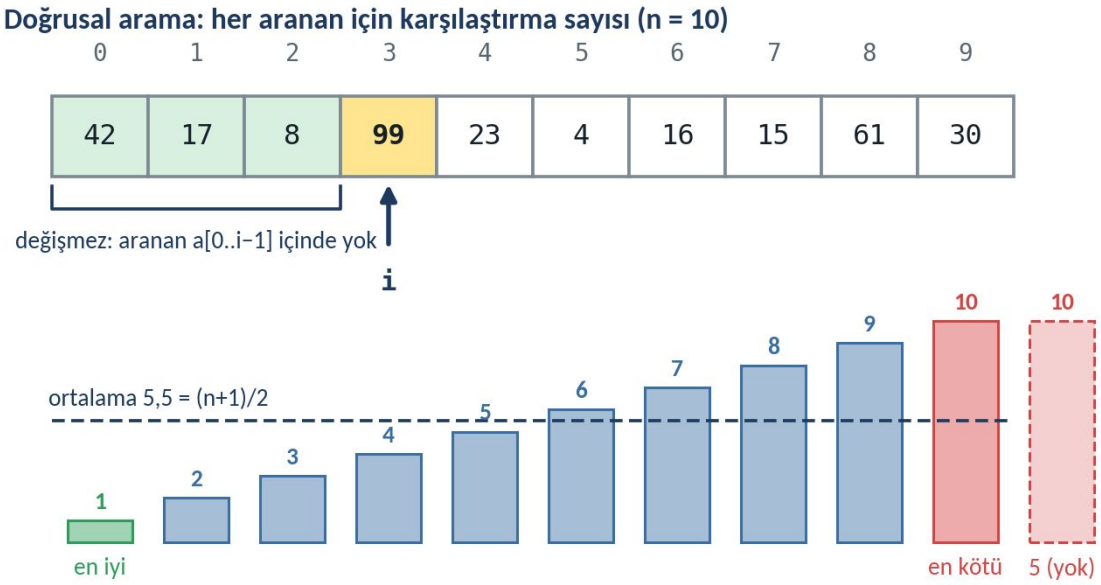
```c
long long dogrusal_toplam(const int *a, size_t n, long long *islem)
{
    long long toplam = 0;
    *islem = 0;
    for (size_t i = 0; i < n; i++) {
        toplam += a[i];
        (*islem)++;                 /* temel işlem: bir toplama */
    }
    return toplam;
}
```

Program n = 0 … 10^5 için iki sayımı da yazdırır: n = 10^5'te 100 000 toplama ve 300 003 ayrıntılı adım. İkisi de n ile **doğrusal** büyür; temel işlem seçimi yalnızca sabit çarpanı değiştirir. İç içe döngülerin sayımı çoğu zaman bir **seriye** dönüşür; bu hafta iki kapalı biçim yeter: **aritmetik seri** $1 + 2 + \ldots + n = n(n+1)/2$ ve **geometrik seri** $1 + 2 + 4 + \ldots + 2^k = 2^{k+1} - 1$.

3.2 En iyi, en kötü ve ortalama durum

Aynı n için farklı girdiler farklı sayıda adım gerektirebilir. Doğrusal aramada aranan ilk elemansa **en iyi durum** (best case) 1 karşılaştırma, son elemansa ya da dizide yoksa **en kötü durum** (worst case) n karşılaştırmadır. **Ortalama durum** (average case) bir **olasılık varsayımı** ister: aranan dizideyse ve her konumda eşit olasılıkla bulunuyorsa beklenen karşılaştırma (1 + 2 + … + n)/n = (n + 1)/2'dir (Şekil 2.1).

Şekil 2.1
Doğrusal arama (Örnek 4): her elemanı bulmak için gereken karşılaştırma; ortalama (n + 1)/2 = 5,5



Örnek 4'ün `dogrusal_ara` fonksiyonu H1'deki doğrusal aramaya Örnek 1'deki gibi bir karşılaştırma sayacı ekler. Program n = 1000 için en iyi 1, en kötü 1000, başarılı aramalarda ortalama 500,5 karşılaştırma ölçer. 1..2n aralığından seçilen 1000 rastgele hedefin ancak 494'ü dizidedir; bu karışımda ortalama 765,79 ölçülür, p(n + 1)/2 + (1 − p)n modeli 753,25 verir. Ortalama durum bir **girdi dağılımı** varsayımıdır.

> HATA AVCISI · **Yanılgı: "O, en kötü durum demektir"**
>
> O, Ω ve Θ (3.3) bir **fonksiyon** hakkında sınır bildirir; "durum" hangi fonksiyonu incelediğimizdir. Doğrusal aramanın en kötü durum süresi Θ(n), en iyi durum süresi Θ(1)'dir; bütün girdiler için O(n) ve Ω(1). "Doğrusal arama O(1)'dir" yanlış, "en iyi durumu Θ(1)'dir" doğrudur (Hata Avcısı 2).

3.3 Asimptotik gösterim: O, Ω ve Θ

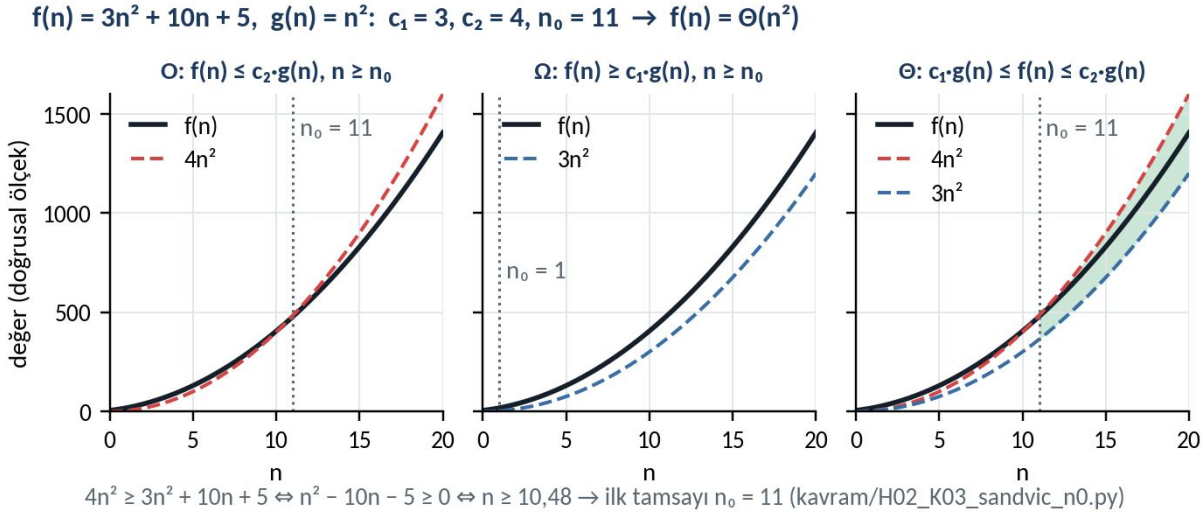
n büyüdükçe T(n)'nin **baskın terimi** her şeyi belirler: $3n^2 + 10n + 5$'te n = 1000 için $3n^2$ = 3 000 000, 10n + 5 = 10 005'tir. Asimptotik gösterim sabit çarpanları ve düşük dereceli terimleri bilerek atar (Cormen vd., 2022; Knuth, 1997):

- **O (üst sınır).** f(n) = O(g(n)): öyle c > 0 ve n_0 vardır ki her n ≥ n_0 için 0 ≤ f(n) ≤ c · g(n).
- **Ω (alt sınır).** f(n) = Ω(g(n)): öyle c > 0 ve n_0 vardır ki her n ≥ n_0 için f(n) ≥ c · g(n) ≥ 0.
- **Θ (sıkı sınır).** f(n) = Θ(g(n)): öyle c_1, c_2 > 0 ve n_0 vardır ki her n ≥ n_0 için 0 ≤ c_1 · g(n) ≤ f(n) ≤ c_2 · g(n); yani f = O(g) **ve** f = Ω(g).

O sıkı olmak zorunda değildir. O yalnızca "en çok bu hızda büyür" der: $3n^2 + 10n + 5 = O(n^2)$ doğrudur, ama $O(n^3)$ ve hatta $O(2^n)$ de doğrudur; ikincileri yazmak bilgi kaybettirir. Benzer biçimde $\Omega(1)$ her pozitif fonksiyon için doğrudur. Bir algoritmanın büyüme sınıfını söylerken alt ve üst sınır aynıysa **Θ** yazarız; O'yu "en az şu kadar" anlamında kullanmak yanlıştır (Alıştırma 2.4c).

"=" burada üyelik demektir: $O(g(n))$ bir fonksiyon **kümesidir**. Örnek: $3n^2 + 10n + 5 = \Theta(n^2)$. Alt sınır için $c_1 = 3$ her $n \geq 1$'de yeter; üst sınır için $c_2 = 4$: $4n^2 \geq 3n^2 + 10n + 5 \Leftrightarrow n^2 - 10n - 5 \geq 0 \Leftrightarrow n \geq 5 + \sqrt{30} \approx 10{,}48$, ilk tamsayı $n_0 = 11$ (Şekil 2.2). `kavram/H02_K03_sandvic_n0.py` aynı sınırı tarayarak bulur ($f(11) = 478 \leq 484 = 4 \cdot 11^2$) ve $f(n)/n^2$ oranının $n = 10$'da 4,05'ten $n = 10^6$'da 3,00001'e indiğini yazar: **limit yaklaşımı**. $\lim f(n)/g(n) = L$ için $0 < L < \infty \Rightarrow f = \Theta(g)$; $L = 0 \Rightarrow f = O(g)$ ama Θ değil; $L = \infty \Rightarrow f = \Omega(g)$ ama Θ değil. Biçimsel kanıt teknikleri Algoritma Analizi dersine kalır; burada tanımı ve kuralları (Tablo 2.2) doğru kullanmak hedeftir.

Şekil 2.2
O, Ω ve Θ tanımları: $f(n) = 3n^2 + 10n + 5$, $n \geq n_0 = 11$ için $3n^2$ ile $4n^2$ arasında kalır



$4n^2 \geq 3n^2 + 10n + 5 \Leftrightarrow n^2 - 10n - 5 \geq 0 \Leftrightarrow n \geq 10{,}48 \rightarrow$ ilk tamsayı $n_0 = 11$ (kavram/H02_K03_sandvic_n0.py)

Tablo 2.2
Asimptotik gösterimde sık kullanılan kurallar (taban belirtilmeyen log ikilik tabandır)

Kural	Örnek
Sabit çarpan atılır; toplamda baskın terim kalır	$5n^2 = \Theta(n^2)$; $n^2 + 100n \log n = \Theta(n^2)$
Polinomun derecesi belirler ($a_k > 0$)	$a_k n^k + \ldots + a_0 = \Theta(n^k)$
Ardışık bloklar toplanır, iç içe bloklar çarpılır	$n + n = \Theta(n)$; $n \cdot n = \Theta(n^2)$
Log tabanı önemsizdir	$\log_3 n = \log_2 n / \log_2 3 = \Theta(\log n)$
log, her pozitif kuvvetten; polinom, üstelden yavaştır	$\log n = O(\sqrt{n})$; $n^{100} = O(2^n)$

3.4 Döngülerden karmaşıklık

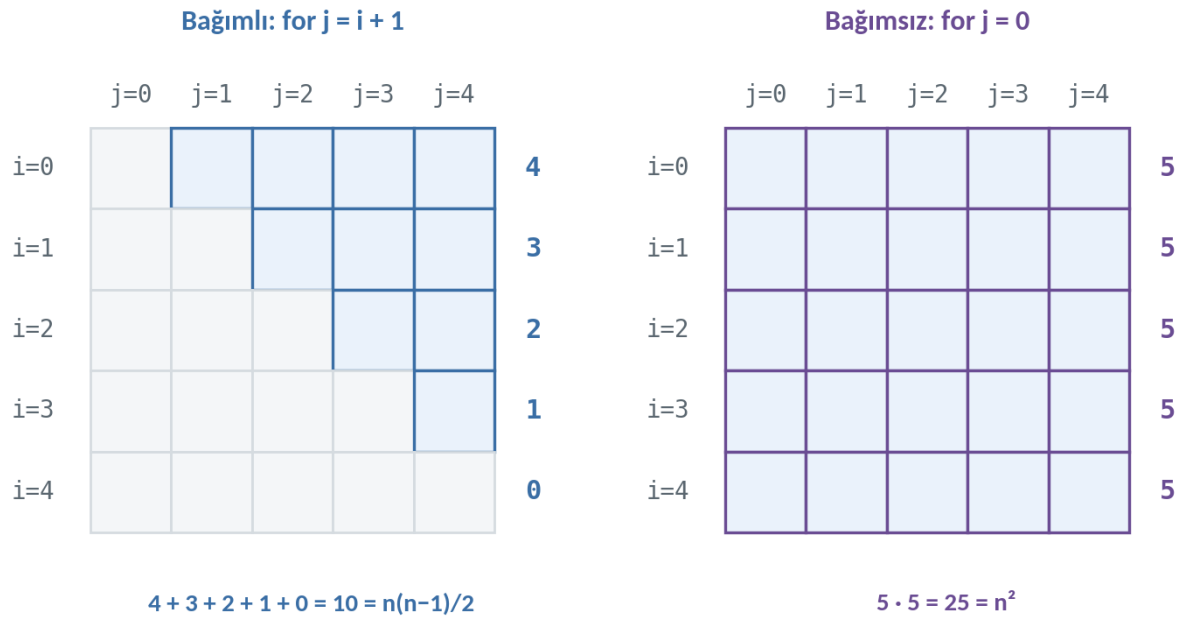
Tablo 2.3 döngü yapısından sınıf çıkarmanın temel desenlerini toplar; her biri bu hafta bir örnekte ya da atölye kartında **sayaçla** doğrulanır.

Tablo 2.3
Döngü desenleri ve büyüme sınıfları

Desen	Yapı	Temel işlem sayısı	Sınıf
Ardışık	for i < n; sonra for j < n	n + n = 2n	Θ(n)
İç içe (bağımsız)	for i < n: for j < n	n · n	Θ(n²)
İç içe (bağımlı)	for i < n: for j = i + 1 … n − 1	(n − 1) + … + 1 + 0 = n(n − 1)/2	Θ(n²)
Yarıya inen	while n > 0: n /= 2	⌊log₂ n⌋ + 1	Θ(log n)
Kare köke kadar	for i = 2; i · i ≤ n	⌊√n⌋ − 1	Θ(√n)
Dış katlanır, iç i'ye kadar	for i = 1; i < n; i *= 2: for j < i	1 + 2 + 4 + … < 2n	Θ(n)

Bağımlı iç içe döngüde iç döngü i. satırda n − 1 − i kez döner; toplam aritmetik bir seridir: n = 5'te 4 + 3 + 2 + 1 + 0 = 10 = n(n − 1)/2 çift, bağımsız sürümde 25 = n² (Şekil 2.3). Bağımsız sürüm her satırda n kez döner. İkisi de Θ(n²)'dir; fark yalnızca sabittir: Örnek 2'nin sayaçları n = 2000'de 1 999 000 ve 4 000 000, oran 2,001. Mini-OBS 2 v2.1'in `yinelenen_bul` fonksiyonu tam olarak bağımlı desendir (Bölüm 10).

Şekil 2.3
Bağımlı iç içe döngünün üçgen işlem diyagramı (n = 5): bağımlı sürümde 4 + 3 + 2 + 1 + 0 = 10 = n(n − 1)/2, bağımsız sürümde 5 · 5 = 25 = n² hücre



n = 5 · satır sonundaki sayı: iç döngünün o satırdaki tur sayısı (n − 1 − i)

C `H02_02_tum_ciftler.c · satır 14–23`

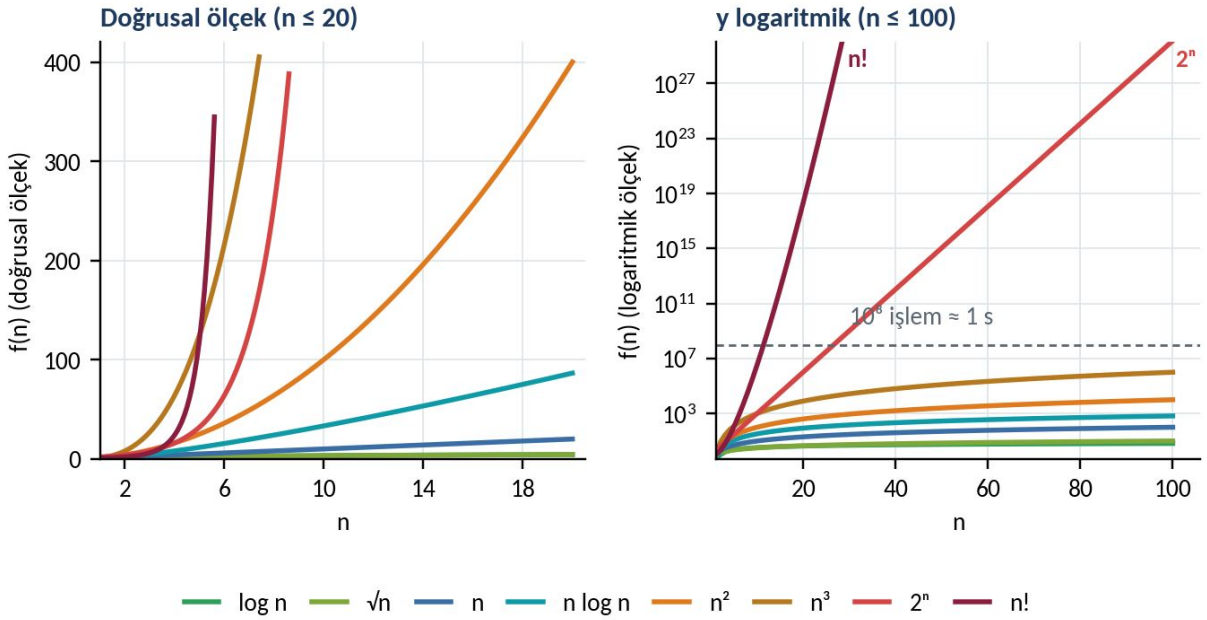
```c
long long tum_ciftler(size_t n)
{
    long long sayac = 0;
    for (size_t i = 0; i < n; i++) {
        for (size_t j = i + 1; j < n; j++) {    /* n - 1 - i kez */
            sayac++;
        }
    }
    return sayac;
}
```

Logaritma sezgisi: "kaç kez yarıya bölebilirim?" Örnek 3'ün `yariya_adim` fonksiyonu gibi n'i her adımda ikiye bölen bir döngü (`n /= 2`) n 0 olana kadar $\lfloor \log_2 n \rfloor + 1$ kez döner: 100 → 50 → 25 → 12 → 6 → 3 → 1 → 0, yedi adım. Örnek 3'ün tablosunda n = 10^6 için 20, n = $2^{64} - 1$ için yalnızca 64 adım; üçe bölen döngüyle oran $\log_2 3 \approx 1{,}58$'e yaklaşır (n = 10^6'da 1,54). $\log_3 n = \log_2 n / \log_2 3$ olduğundan **log tabanı yalnızca sabit bir çarpandır**; Θ(log n) yazarken taban belirtilmez.

3.5 Büyüme sınıfları ve uygulanabilirlik

Algoritmaların çoğu şu sınıflardan birine düşer: 1, log n, √n, n, n log n, n^2, n^3, 2^n, n!. Şekil 2.4 bunları solda doğrusal, sağda logaritmik y ekseninde gösterir: doğrusal ölçekte n ≤ 20'de bile n!, n^3 ve 2^n grafikten çıkar, log n, √n ve n yatay eksene yapışır; logaritmik ölçek hepsini aynı kâğıda sığdırır. Örnek 6 aynı karşılaştırmayı sayılarla yapar ve `double`'a sığmayan değerleri 10^k biçiminde yazar (Bölüm 5.2).

Şekil 2.4
Büyüme sınıfları iki ölçekte: solda doğrusal ölçek (n ≤ 20), sağda y logaritmik ölçek (n ≤ 100); kesikli çizgi: 10^8 temel işlem ≈ 1 saniye varsayımı



Tersinden soralım: saniyede 10^8 temel işlem yapan bir makinede **1 saniyede çözülebilecek en büyük n** nedir? Bu **uygulanabilirlik** (feasibility) tablosu (Tablo 2.4) problem boyutundan kabul edilebilir algoritma sınıfını geri çıkarmamızı sağlar (Bölüm 9).

Tablo 2.4
1 saniyede çözülebilecek en büyük n (Örnek 6'nın yöntemiyle; 1 ve log n tabloda yok: log n için sınır 2^(işlem sayısı), pratikte sınırsız)

Sınıf	10^6 işlem/s	10^8 işlem/s	10^9 işlem/s
√n	1 000 000 000 000	10 000 000 000 000 000	1 000 000 000 000 000 000
n	1 000 000	100 000 000	1 000 000 000
n log n	62 746	4 523 071	39 620 077
n^2	1 000	10 000	31 622
n^3	100	464	1 000

Sınıf	10^6 işlem/s	10^8 işlem/s	10^9 işlem/s
2^n	19	26	29
n!	9	11	12

> **GERÇEK DÜNYADAN · Büyük Soru'nun yanıtı**
>
> n = 10^6'da 1 saniye süren bir program n = 10^9'da (girdi 1000 kat) **doğrusal** ise ≈ 1000 s (≈ 16,7 dakika), **n log n** ise ≈ 1500 s (≈ 25 dakika), **karesel** ise ≈ 10^6 s, yani ≈ 11,6 gün sürer. Haftanın Deneyi'nde aynı algoritmanın Python sürümü C sürümünden bu makinede 31–104 kat yavaş ölçüldü (Bölüm 6): dil seçimi süreyi **bir sabitle** çarpar. Girdi 1000 kat büyüyünce süre ise sınıfa göre 10^3 (doğrusal) ile 10^6 (karesel) kat artar ve bu fark n ile büyümeye devam eder. Önce sınıf, sonra sabit.

3.6 Deneysel analiz: oran testi ve log–log eğim

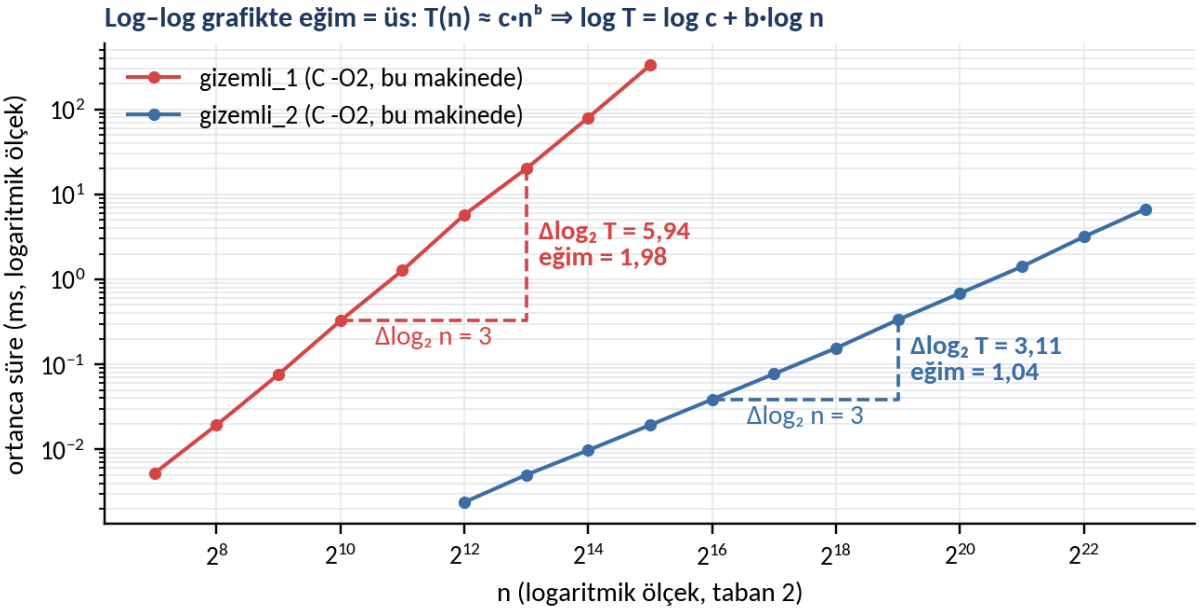
Kuramı ölçümle sınamanın en basit yolu **iki katına çıkarma oran testidir** (doubling ratio test; Sedgewick & Wayne, 2011): $T(n) \approx c \cdot n^b$ ise $T(2n)/T(n) \approx 2^b$ olur; oran ≈ 2 doğrusal, ≈ 4 karesel, ≈ 8 kübik. Aynı bilgiyi **log–log grafik** verir: $\log T = \log c + b \cdot \log n$ bir doğrudur; eğimi üs b'dir, c doğruyu yalnızca yukarı aşağı kaydırır. Eğim grafikten iki noktayla okunur: $b \approx \Delta\log_2 T / \Delta\log_2 n$ (Şekil 2.5; üç dilin karşılaştırması Bölüm 6, Şekil 2.14). Yöntemin adımları Şekil 2.6'dedir.

C **H02_07_oran_testi.c · satır 21–34**

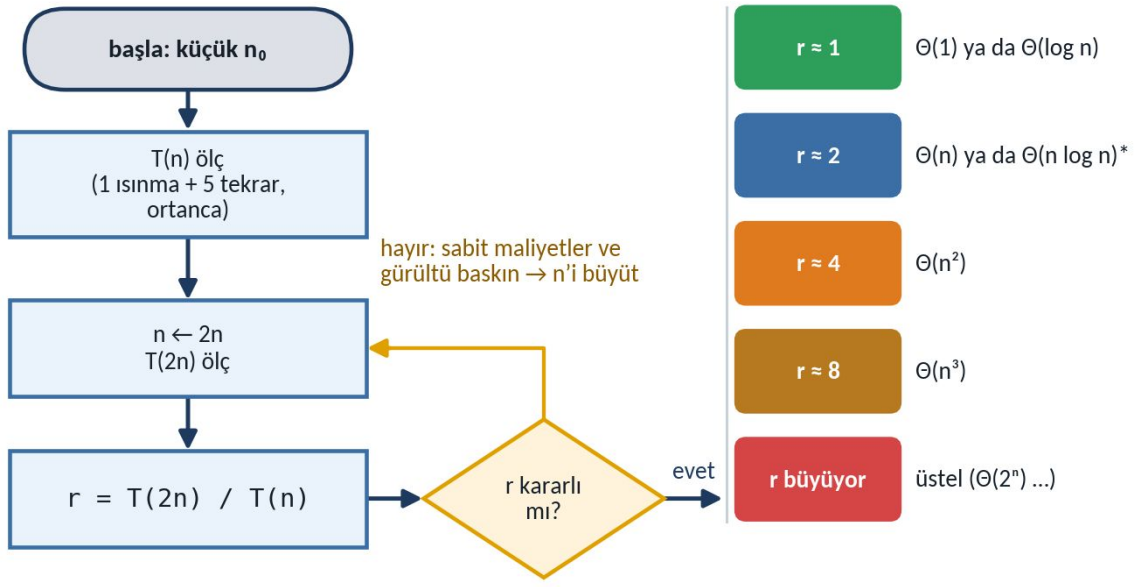
```c
long long iki_toplam(const int *a, size_t n, long long *kar)
{
    long long sayi = 0, k = 0;
    for (size_t i = 0; i + 1 < n; i++) {
        for (size_t j = i + 1; j < n; j++) {
            k++;
            if (a[i] + a[j] == 0) {
                sayi++;
            }
        }
    }
    *kar = k;
    return sayi;
}
```

Örnek 7'nin sayaç oranları n = 500, 1000, 2000'de 4,008; 4,004; 4,002: kuram $T(2n)/T(n) = 2(2n-1)/(n-1) \to 4$'ü birebir verir. Süre oranı gürültülüdür; bu makinede son katlamada (n = 2000) C 4,37, C++ 4,20, Python 4,19 ölçüldü. Üç kural: (1) n'i **katlayın** ve en küçük n'leri yorumdan ayırın; (2) her n için **ortancayı** kullanın; (3) sınıfı tek bir orandan değil, oranların gidişinden ve eğimden okuyun. Oranı 2 olan bir fonksiyon Θ(n) ya da Θ(n log n) olabilir: n log n'de oran $2 \cdot \log(2n)/\log n$, $n = 2^{16}$'da yalnızca 2,13'tür; ayırt etmek için T(n)/n'in düz kalıp kalmadığına bakılır (Bölüm 6).

Şekil 2.5
Log–log grafikte eğimden üs okuma (Haftanın Deneyi, C -O2, bu makinede): üç katlamalık aralıkta Δlog₂ T / Δlog₂ n gizemli_1'de ≈ 2, gizemli_2'de ≈ 1



Şekil 2.6
Oran testinin adımları: küçük n_0'dan başla, her n için ortancayı ölç, n'i katla ve oranı sınıfa çevir; oran kararlı değilse n'i büyüt



* n ile n log n'i ayırmak için T(n)/n'in düz kalıp kalmadığına bakın; ek olarak log–log eğim ≈ $\log_2 r$

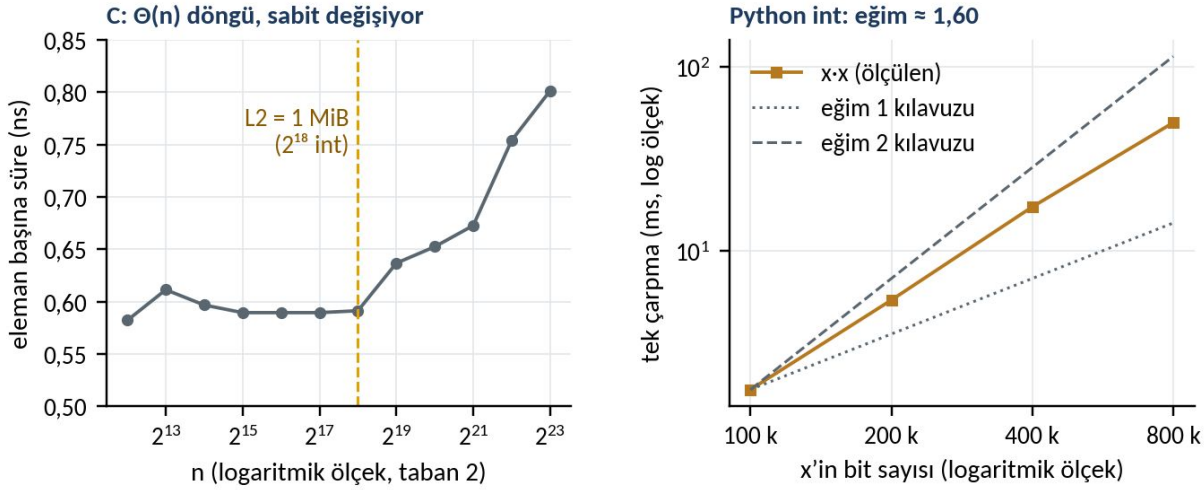
3.7 Bellek karmaşıklığı ve Kaputun Altında

Aynı gösterim belleğe de uygulanır: **ek bellek** (auxiliary space) girdi dışında kullanılan alandır. Doğrusal arama O(1) ek bellek kullanır; Eratosthenes kalburu N + 1 baytlık bir işaret dizisi ister, O(N), ama karşılığında deneme bölmesinden kat kat az işlem yapar (Bölüm 4.2). Daha çok bellekle daha az zaman seçimine **zaman–bellek ödünleşimi** (time–memory trade-off) denir.

> **KAPUTUN ALTINDA · 1 · Birim maliyetin sınırı: aynı Θ(n), değişen sabit**
>
> RAM modeli her dizi erişimini birim maliyetli sayar; işlemci ise veriyi önbellekten (L1, L2, L3) ya da ana bellekten getirir. Haftanın Deneyi'ndeki doğrusal gizemli_2'nin C -O2 sürümünde eleman başına süre $n = 2^{16}$'da 0,59 ns iken dizi çekirdek başına 1 MiB'lık L2 önbelleğini aştıktan sonra 2^{21}'de 0,67, 2^{23}'te 0,80 ns oldu: eğim ≈ 1 kaldı, sabit değişti. AP-1 H11'deki satır/sütun deneyi aynı olgunun başka yüzüydü; dal tahmini (branch prediction) de benzer fark yaratır.

> **KAPUTUN ALTINDA · 2 · "Sabit zamanlı işlem" Python'un büyük tamsayılarında geçerli değildir**
>
> Python `int`'i sınırsızdır; k bitlik bir sayıyı ikiye katlamak O(k) bit işlemidir. Örnek 12'de 2^n'i n kez ikiye katlamak $n = 10^4$'te 2,35 ms, $n = 10^5$'te 176,8 ms sürdü: n 10 kat, süre ≈ 75 kat (toplam $O(n^2)$ bit işlemi). Tek bir $x \cdot x$ çarpması x 10^5 bitten $8 \cdot 10^5$ bite çıkınca 1,78 ms'den 49,74 ms'ye uzadı, log–log eğim ≈ 1,60: CPython büyük sayılarda Karatsuba çarpmasını kullanır (kuramsal üs $\log_2 3 \approx 1{,}585$). C'deki `uint64_t` sabit maliyetlidir ama 64 bitte sarar (Bölüm 5.4). Üçüncü kutu kalburun belleğidir (Bölüm 4.2).

Şekil 2.7
Kaputun Altında 1–2 (bu makinede): solda doğrusal gizemli_2'de eleman başına süre L2 önbelleği aşılınca artar (eğim ≈ 1 kalır); sağda Python'da tek x · x çarpmasının süresi bit sayısıyla büyür (log–log eğim ≈ 1,6)



4. Haftanın Algoritmaları: Sayı Kuramından Alt Dizilere

Dört **amiral** algoritmayı standart anlatım sırasıyla işliyoruz: 1 problem → 2 fikir → 3 elle izleme → 4 sözde kod → 5 doğruluk → 6 analiz ve kimlik kartı → 7 C → 8 üç dil → 9 test → 10 deney → 11 varyantlar. Kodlar `kodlar/c/`, `kodlar/cpp/` ve `kodlar/python/` dosyalarından aynen alınmıştır; satır numaraları dosyalarla aynıdır.

4.1 Deneme bölmesiyle asallık: √n sınırı

1 · Problem. n ≥ 0 bir tamsayı **asal** mı (tam iki pozitif böleni var mı)? 97 asaldır; 91 = 7 · 13 bileşiktir. **2 · Fikir.** Saf yöntem 2 … n − 1 adaylarını dener: n − 2 bölme. Ama n bileşikse n = d · e ($2 \le d \le e$) ve d · d ≤ n olur: **en küçük bölen √n'i aşamaz**. Adayları i · i ≤ n iken denemek yeter. **3 · Elle izleme.** n = 91: i = 2, 3, 4, 5, 6, 7 için 91 mod i = 1, 1, 3, 1, 1, 0; i = 7 bölünce döngü erken biter. n = 97'de 2 … 9 denenir, 10 · 10 > 97 olunca asal olduğu kesinleşir (8 bölme; Şekil 2.8).

Şekil 2.8
Deneme bölmesinin elle izlemesi: adaylar yalnızca i · i ≤ n iken denenir; n = 91'de 7 böler ve döngü erken biter, n = 97'de 2 … 9 denenir ve hiçbiri bölmez



Neden √n yeter? n = d · e ise d ile e'nin ikisi birden √n'den büyük olamaz: 36 = 2·18 = 3·12 = 4·9 = 6·6.

Asal n'de ⌊√n⌋ − 1 bölme: n = 7919 için 87 (Örnek 8).

SÖZDE KOD · 4 · Sözde kod: deneme bölmesi

```
algoritma ASAL-MI(n)
    eğer n < 2 ise döndür YANLIŞ
    i ← 2
    i · i ≤ n iken tekrarla
        eğer n mod i = 0 ise döndür YANLIŞ     ▷ i böler
        i ← i + 1
    döndür DOĞRU                               ▷ √n'e kadar bölen yok
```

5 · Doğruluk. Değişmez: *döngü başında 2 ≤ d < i olan hiçbir d n'i bölmez*. Başlatma: i = 2'de böyle d yok. Koruma: n mod i ≠ 0 ise i artar. Sonlanma: i · i > n ise n bileşik olsaydı en küçük böleni d ≤ √n < i olurdu, değişmezle çelişir. Döngü erken biterse i n'i böler. **6 · Analiz.** En iyi durum çift n: 1 bölme. En kötü durum asal n: ⌊√n⌋ − 1 bölme, O(√n). Ek bellek O(1). Girdi boyutu bit sayısı b ise √n ≈ 2^(b/2): bit sayısına göre üstel (Merak Kutusu).

Tablo 2.5
Deneme bölmesiyle asallık: algoritma kimlik kartı

ALGORİTMA KİMLİK KARTI	**Deneme bölmesiyle asallık** *(primality test by trial division)*
Strateji	Kaba kuvvet; adaylar 2 … ⌊√n⌋
Girdi önkoşulu	n ≥ 0; n < 2^{63} (`uint64_t`'de `i * i` taşmasın)
Zaman (en iyi · ortalama · en kötü)	O(1) · — · O(√n)
Ek bellek	O(1)
Özellikler	Erken çıkış: ilk bölende durur (çift n'de 1 bölme). Asal n'de ⌊√n⌋ − 1 bölme (n = 10^9 + 7 için 31 621). Girdinin bit sayısına göre üsteldir: b bitlik n için ≈ 2^(b/2) bölme.
Değişmez / doğruluk	Döngü başında 2 ≤ d < i olan hiçbir d, n'i bölmez
Kütüphane karşılığı	C: yok (elle) · C++: yok · Python: yok (`sympy.isprime` üçüncü taraf)

Ne zaman kullanılır	Tek tek sorgulanan, yaklaşık 10^{12}'ye kadar sayılar; ek bellek istenmiyorsa
Ne zaman kullanılmaz	Bir aralıktaki tüm sayılar (kalbur daha hızlı); çok büyük sayılar (olasılıksal testler, ör. Miller–Rabin)

7 · C. Sınır `sqrt(n)` ile değil `i * i <= n` ile yazılır (büyük n'de kayan nokta ⌊√n⌋'i yanlış bulabilir); n < 2^{63} ön koşulu `i * i`'nin taşmamasını güvenceye alır. **8 · Üç dil.**

C `H02_08_deneme_bolmesi.c · satır 22–35`

```c
bool asal_mi(uint64_t n, uint64_t *bolme)
{
    *bolme = 0;
    if (n < 2) {
        return false;
    }
    for (uint64_t i = 2; i * i <= n; i++) {
        (*bolme)++;
        if (n % i == 0) {
            return false;   /* i böler */
        }
    }
    return true;
}
```

C++ `H02_08_deneme_bolmesi.cpp · satır 29–35`

```cpp
    for (u64 i = 2; i * i <= n; i++) {
        bolme++;
        if (n % i == 0) {
            return false;       // i böler
        }
    }
    return true;
```

Python `H02_08_deneme_bolmesi.py · satır 19–25`

```python
    i = 2
    while i * i <= n:
        bolme += 1
        if n % i == 0:
            return False, bolme  # i böler
        i += 1
    return True, bolme
```

> DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C ve C++'ta `i * i <= n` koşulu `uint64_t` ile ve n < 2^63 ön koşuluyla taşmasızdır; Python tamsayıları taşmadığından aynı koşul her n için güvenlidir. Sınır `sqrt(n)` ile hesaplanmaz: büyük n'de kayan nokta yuvarlaması ⌊√n⌋'i bir eksik ya da fazla bulabilir. Python sürümü sayacı demetle döndürür (`return True, bolme`).

9 · Test. Saf sürüm (2 … n − 1) **kâhindir**: n ≤ 2 · 10^6 satırlarında iki sürümün kararı karşılaştırılır. Sınır durumları 0, 1, 2 (döngü hiç dönmez), tam kareler 4 ve 25 (<= eşitliği kapsamalı), büyük asallar ve yarı asal. **10 · Deney.** Örnek 8'in sayaçları kuramla birebirdir: asal n'de ⌊√n⌋ − 1 bölme (7919 → 87, 10^9 + 7 → 31 621, 999 999 999 989 → 999 998), saf sürümde n − 2 (7919 → 7 917); yarı asal 1 000 036 000 099 = 1 000 003 · 1 000 033 için 1 000 002 bölme. Tek tek asallık testi **çok sayıda** sayı için pahalıdır: 2 … 10^7 aralığının tamamına uygulanınca 1 746 210 133 bölme yapar (Bölüm 11.5). **11 · Varyantlar.** 2'den sonra yalnızca tek adaylar (bölmeler yarıya iner); 6k ± 1 biçimli adaylar (üçte bire); yalnızca asal adaylarla bölmek için önce asalları üretmek: kalbur.

4.2 Eratosthenes kalburu: zaman–bellek ödünleşimi

1 · Problem. 2 … N arasındaki **bütün** asalları bulmak ya da saymak (π(N); π(100) = 25). **2 · Fikir.** Tek tek sınamak yerine bileşikleri eleriz: 2 asaldır, katlarını işaretle; işaretsiz kalan ilk sayı (3) asaldır, onun katlarını işaretle… İki iyileştirme: p'nin katlarını **p · p'den** başlatmak (daha küçük katlar daha küçük bir asalca zaten işaretlenmiştir) ve dış döngüyü **p · p ≤ N** iken sürdürmek. **3 · Elle izleme.** N = 50'de dış döngü i = 2 … 7 için döner (8 · 8 > 50); bunlardan asal olan dördü tur yapar: p = 2, 3, 5, 7 (Şekil 2.9; 7 · 7 = 49 ≤ 50, sonraki asal 11 için 11 · 11 > 50). p = 3 turunda 7 sayı ilk kez işaretlenir, 7 sayı (12, 18, …, 48) 2 tarafından zaten işaretlenmiştir. Sonuç: 15 asal, 45 işaretleme; iç döngüsü 2i'den başlayan ve dış döngüsü N'ye kadar süren sürüm (Hata Avcısı 6) 62 işaretleme yapar.

SÖZDE KOD · 4 · Sözde kod: Eratosthenes kalburu

```
algoritma KALBUR(N)
    bileşik[0..N] ← YANLIŞ
    i ← 2
    i · i ≤ N iken tekrarla
        eğer bileşik[i] = YANLIŞ ise          ▷ i asal
            j ← i · i
            j ≤ N iken tekrarla
                bileşik[j] ← DOĞRU;  j ← j + i
        i ← i + 1
    döndür { k : 2 ≤ k ≤ N ve bileşik[k] = YANLIŞ }
```

5 · Doğruluk. Değişmez: *i. turun başında, en küçük asal böleni i'den küçük olan her bileşik işaretlidir.* Başlatma (i = 2): böyle sayı yok. Koruma: i işaretsizse i'den küçük hiçbir asal i'yi bölmez, i asaldır; tur, en küçük asal böleni i olan bileşikleri (en küçüğü i · i) işaretler. Sonlanma: i · i > N iken N'ye kadar her bileşiğin en küçük asal böleni ≤ √N < i'dir. **6 · Analiz.** p'nin turu ≈ N/p işaretleme yapar; asalların terslerinin toplamı log log N gibi çok yavaş büyür: Θ(N log log N). Örnek 9'da işaretleme/N oranı N = 10'da 0,50'den N = 10^5'te 1,93'e ancak çıkar. Ek bellek N + 1 bayt, Θ(N): deneme bölmesiyle karşılaştırma bir **zaman–bellek ödünleşimidir**.

Tablo 2.6
Eratosthenes kalburu: algoritma kimlik kartı

ALGORİTMA KİMLİK KARTI	**Eratosthenes kalburu** *(sieve of Eratosthenes)*
Strateji	Ön hesaplama; zaman–bellek ödünleşimi
Girdi önkoşulu	N önceden bilinir; N + 1 bayt belleğe sığar
Zaman (en iyi · ortalama · en kötü)	— · — · O(N log log N)
Ek bellek	O(N)

Özellikler	Katlar p·p'den başlar, dış döngü i·i ≤ N'de durur. N = 10^5'te 193 078 işaretleme (deneme bölmesi: 2 745 694 bölme). N = 10^8 için bayt dizisi ≈ 100 MB, bit dizisi ≈ 12,5 MB.
Değişmez / doğruluk	i. turun başında, en küçük asal böleni i'den küçük olan her bileşik sayı işaretlidir; bu yüzden işaretsiz i asaldır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`sympy.sieve` üçüncü taraf)
Ne zaman kullanılır	Bir aralıktaki tüm asallar, çok sayıda asallık sorgusu (N ≈ 10^7–10^8)
Ne zaman kullanılmaz	Tek bir büyük sayının asallığı; N + 1 bayt belleğe sığmıyorsa (parçalı kalbur gerekir)

Şekil 2.9
Kalbur ızgarası animasyon kareleri (N = 50, programla üretilmiş izleme): yeni işaretlenenler sarı, yeniden yazılanlar taralı; p · p başlangıcı kesikli çerçeveli



7 · C. `i * i <= n` dış döngüyü √N'de durdurur, `j = i * i` iç döngüyü başlatır. İkisini unutmak doğruluğu bozmaz, işi artırır (Hata Avcısı 6); `int` ile yazmak taşmaya yol açar (Hata Avcısı 5). **8 · Üç dil** (işaret dizisi ve iç içe döngü).

C **H02_09_kalbur.c · satır 23–40**

```c
long kalbur(size_t n, unsigned char *bilesik, long long *isaretleme)
{
    long asal = 0;
    size_t i, j;
    *isaretleme = 0;
    for (i = 0; i <= n; i++) {
        bilesik[i] = 0;
    }
    /* i asalsa katlarını i*i'den başlayarak işaretle: daha küçük
     * katlar, daha küçük bir asal tarafından zaten işaretlendi */
    for (i = 2; i * i <= n; i++) {
        if (!bilesik[i]) {
            for (j = i * i; j <= n; j += i) {
                bilesik[j] = 1;
                (*isaretleme)++;
            }
        }
    }
```

C++ **H02_09_kalbur.cpp · satır 28–38**

```cpp
    bilesik.assign(n + 1, 0);           // n + 1 bayt; vector<bool> değil
    // i asalsa katlarını i*i'den başlayarak işaretle: daha küçük
    // katlar, daha küçük bir asal tarafından zaten işaretlendi
    for (i = 2; i * i <= n; i++) {
        if (!bilesik[i]) {
            for (j = i * i; j <= n; j += i) {
                bilesik[j] = 1;
                isaretleme++;
            }
        }
    }
```

Python **H02_09_kalbur.py · satır 19–28**

```python
    bilesik = bytearray(n + 1)          # n + 1 bayt, hepsi 0
    # i asalsa katlarını i*i'den başlayarak işaretle: daha küçük
    # katlar, daha küçük bir asal tarafından zaten işaretlendi
    i = 2
    while i * i <= n:
        if not bilesik[i]:
            for j in range(i * i, n + 1, i):
                bilesik[j] = 1
                isaretleme += 1
        i += 1
```

DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**

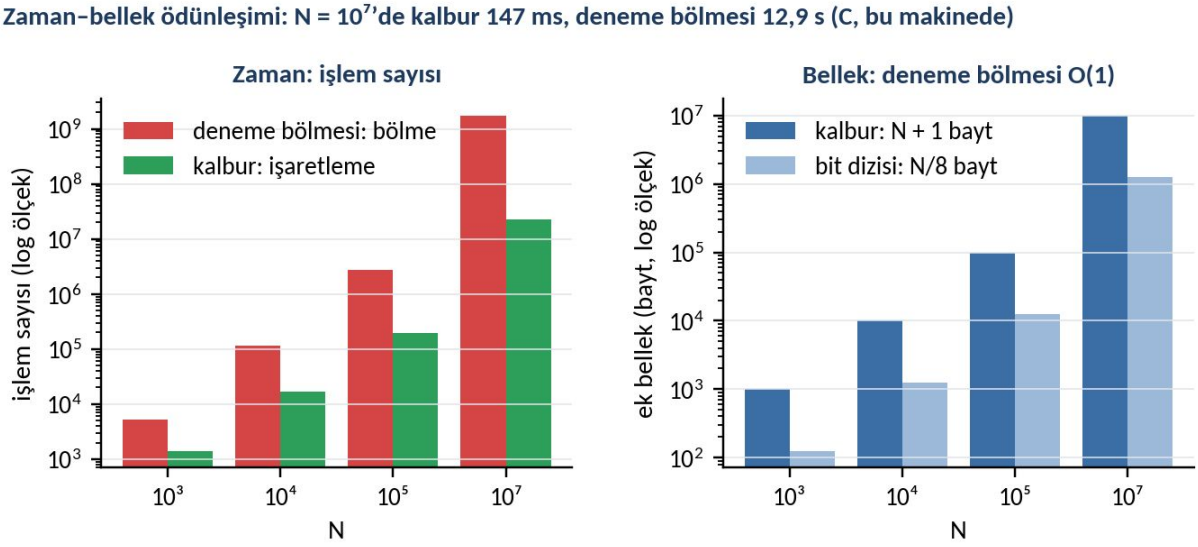
Kalbur dizisi C'de `malloc(n + 1)`, C++'ta `std::vector<unsigned char>`, Python'da `bytearray(n + 1)` ile eleman başına 1 bayt tutulur. C++'ta `std::vector<bool>` bit düzeyinde paketlenir (≈ n/8 bayt) ama erişimi daha yavaştır; Python `list` ise eleman başına 8 baytlık işaretçi saklar. Python'da `range(i * i, n + 1, i)` adımlı aralığı C'deki `j += i` döngüsünün karşılığıdır.

9 · Test. Kâhin Örnek 8'in deneme bölmesidir: her N için iki yöntemin asal sayısı aynı çıkmalıdır (çıktıdaki "π(N)" ve "deneme" sütunları); bilinen değerler de kâhindir: π(10) = 4, π(100) = 25, π(10^7) = 664 579. **10 · Deney.** Bu makinede C'de N = 10^5 için kalbur 0,52 ms, deneme bölmesi 20,0 ms; N = 10^7'de 147 ms'ye karşı 12,9 s (22 850 051 işaretleme, 1 746 210 133 bölme). Python'da N = 10^7 kalburu 2,27 s (Şekil 2.10; ayrıntı Bölüm 11.5).

ÇIKTI (ÜÇ DİLDE AYNI)

```
$ ./H02_09_kalbur
         N     π(N)   deneme        bölme  işaretleme  işar./N  bellek(B)
        10        4        4            8           5    0.500          11
       100       25       25          236         104    1.040         101
      1000      168      168         5288        1411    1.411        1001
     10000     1229     1229       117527       16981    1.698       10001
    100000     9592     9592      2745694      193078    1.931      100001
```

Şekil 2.10
Zaman–bellek ödünleşimi (C, bu makinede): kalbur deneme bölmesinden çok daha az işlem yapar ama N + 1 bayt ek bellek ister; bit dizisi belleği sekizde birine indirir



KAPUTUN ALTINDA · **Kaputun Altında 3 · Kalburun bellek gereksinimi**

N = 10^8 için bayt başına bir işaret ≈ 100 MB demektir. Her işareti bir **bit** ile tutmak belleği sekizde birine (≈ 12,5 MB), yalnızca tek sayıları saklamak bir yarıya daha indirir (≈ 6,25 MB): 2 dışındaki çift sayılar zaten bileşiktir. Bedeli her erişimdeki bit kaydırma ve maskelemedir.

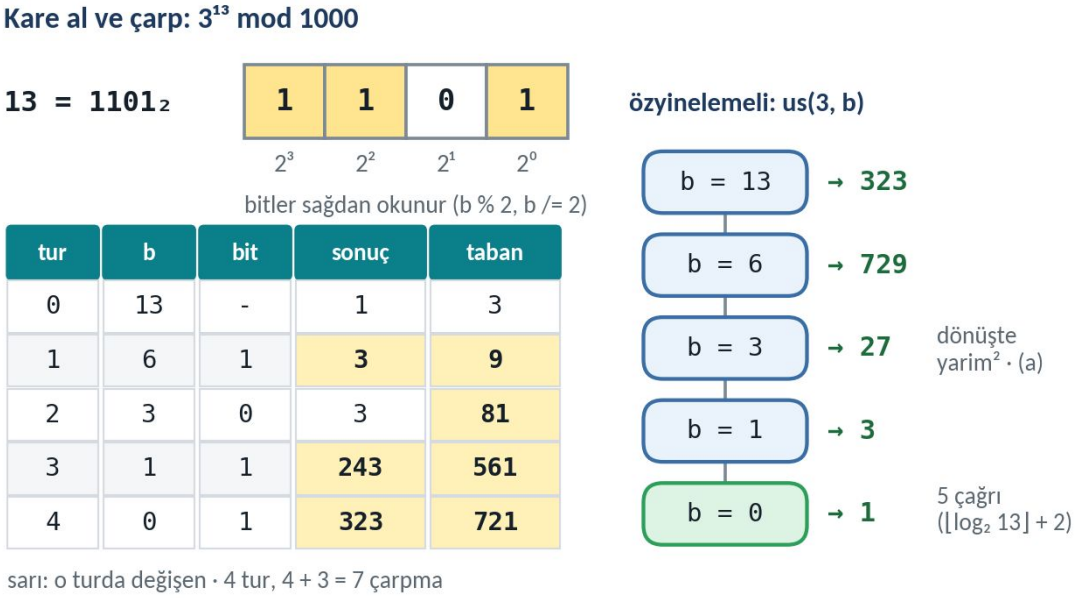
11 · Varyantlar. Bit dizisi ve yalnızca tek sayılar (yukarıdaki kutu; Alıştırma 2.13); belleğe sığmayan N için aralığı √N boyutlu bloklarla işleyen **parçalı kalbur** (segmented sieve); her bileşiği bir kez işaretleyen doğrusal kalbur (Θ(N), ek bir asal listesiyle).

4.3 Hızlı üs alma ve modüler üs: kare al ve çarp

1 · Problem. a^b mod m; b 10^{18} kadar büyük olabilir (3^{13} mod 1000 = 323). Saf yöntem b çarpma yapar: b = 10^{18} için saniyede 10^9 çarpmayla ≈ 30 yıl. **2 · Fikir.** Üssü yarıya indiririz: b çiftse a^b = (a^(b/2))², tekse a^b = a · (a^((b−1)/2))²; ⌊$\log_2$ b⌋ + 1 adım yeter: **böl ve fethetin** (divide and conquer) ilk tadı. Yinelemeli

biçimde b'nin bitleri sağdan okunur: bit 1 ise sonuç tabanla çarpılır, her adımda tabanın karesi alınır. **3 · Elle izleme.** $13 = 1101_2$; başlangıçta sonuç = 1, taban = 3 (Şekil 2.11): her turda son bit okunur, bit 1 ise sonuç tabanla çarpılır, ardından tabanın karesi alınır ve b yarıya iner. Yalnızca bit 1 olan turlarda sonuç değişir ($3 \to 3 \to 243 \to 323$); dört turda $3^{13} \bmod 1000 = 323$ (Örnek 11'in izleme çıktısı).

Şekil 2.11
Kare al ve çarp: $3^{13} \bmod 1000$. Bitler sağdan okunur; sarı hücreler o turda değişen değerlerdir (4 tur, 4 + 3 = 7 çarpma). Sağda özyinelemeli sürümün çağrıları: $\lfloor \log_2 13 \rfloor + 2 = 5$ çağrı



tur	b	bit	sonuç	taban
0	13	-	1	3
1	6	1	**3**	**9**
2	3	0	3	**81**
3	1	1	**243**	**561**
4	0	1	**323**	**721**

SÖZDE KOD · 4 · Sözde kod: yinelemeli kare al ve çarp

```
algoritma ÜS-MOD(a, b, m)              ▷ ön koşul 1 ≤ m < 2^32
    sonuç ← 1 mod m;  taban ← a mod m
    b > 0 iken tekrarla
        eğer b tek ise sonuç ← sonuç · taban mod m
        taban ← taban · taban mod m          ▷ kare al
        b ← ⌊b / 2⌋                          ▷ sonraki bit
    döndür sonuç
```

5 · Doğruluk. b_0 başlangıçtaki üs olsun. Değişmez: *her turun başında sonuç · taban$^b \equiv a^{(b_0)} \pmod m$*. Başlatma: $1 \cdot a^{(b_0)}$. Koruma: b tekse sonuç′ = sonuç · taban, taban′ = taban², b′ = (b − 1)/2 ve sonuç′ · taban′$^{(b')}$ = sonuç · taban^b; b çiftse sonuç değişmez, taban′$^{(b/2)}$ = taban^b. Sonlanma: b = 0'da sonuç $\equiv a^{(b_0)}$. **6 · Analiz.** Tur sayısı bit uzunluğudur; çarpma = bit uzunluğu + 1 bitlerinin sayısı $\le 2\lfloor \log_2 b \rfloor + 2$: $O(\log b)$. Sayaçla: b = 13 → 4 + 3 = 7; b = 1 000 → 10 + 6 = 16; $b = 10^{18}$ → 60 + 24 = 84. Yinelemeli sürüm $O(1)$, özyinelemeli sürüm $\lfloor \log_2 b \rfloor + 2$ derinlikte çağrı yığını kullanır.

Tablo 2.7
Hızlı üs alma: algoritma kimlik kartı

ALGORİTMA KİMLİK KARTI **Hızlı üs alma ($a^b \bmod m$)** *(modular exponentiation by squaring)*	
Strateji	Böl ve fethet: kare al ve çarp
Girdi önkoşulu	$b \ge 0$; $1 \le m < 2^{32}$: $(m-1)^2 < 2^{64}$
Zaman (en iyi · ortalama · en kötü)	$O(\log b)$ · $O(\log b)$ · $O(\log b)$

Ek bellek	O(1) / O(log b)
Özellikler	Çarpma sayısı = b'nin bit uzunluğu + 1 bitlerinin sayısı ≤ $2\lfloor \log_2 b \rfloor + 2$ ($b = 10^{18}$ için 84; saf yöntem 10^{18}). Özyinelemeli sürüm log b derinlikte çağrı yığını kullanır. RSA ve Fermat testinin temel işlemi.
Değişmez / doğruluk	Yinelemeli sürümde her turun başında: sonuc · taban^b ≡ a^(başlangıçtaki b) (mod m)
Kütüphane karşılığı	C: yok (`pow()` `double` döndürür, mod almaz) · C++: yok · Python: `pow(a, b, m)`
Ne zaman kullanılır	Büyük üsler ($b \approx 10^{18}$), modüler aritmetik, kriptografi, matris üssüyle yineleme bağıntıları
Ne zaman kullanılmaz	Küçük sabit üsler (`x * x * x` daha açık); $m \geq 2^{32}$ iken taşmasız çarpma yoksa

7 • C. Taşma denetimi algoritmanın parçasıdır: çarpanlar m'den küçük olduğundan çarpım en çok $(m-1)^2$ ve $m < 2^{32}$ ise $(m-1)^2 < 2^{64}$ olur. Daha büyük m'de `a * b % m` sessizce yanlış sonuç verir (Hata Avcısı 7); `mod_gecerli` ön koşulu denetler. **8 • Üç dil.**

C **H02_11_hizli_us.c · satır 27–42**

```c
uint64_t us_mod(uint64_t a, uint64_t b, uint64_t m, int *carpma)
{
    uint64_t sonuc = 1 % m;
    a %= m;
    *carpma = 0;
    while (b > 0) {
        if (b % 2 == 1) {   /* son bit 1 */
            sonuc = sonuc * a % m;
            (*carpma)++;
        }
        a = a * a % m;      /* kare al */
        (*carpma)++;
        b /= 2;
    }
    return sonuc;
}
```

C++ **H02_11_hizli_us.cpp · satır 34–42**

```cpp
    while (b > 0) {
        if (b % 2 == 1) {   // son bit 1
            sonuc = sonuc * a % m;
            carpma++;
        }
        a = a * a % m;      // kare al
        carpma++;
        b /= 2;
    }
```

Python **H02_11_hizli_us.py · satır 25–31**

```python
    while b > 0:
        if b % 2 == 1:      # son bit 1
            sonuc = sonuc * a % m
            carpma += 1
        a = a * a % m       # kare al
        carpma += 1
        b //= 2
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C ve C++'ta `uint64_t` çarpımının taşmaması için m < 2^32 ön koşulu gerekir: $(m-1)^2 < 2^{64}$. Python tamsayıları taşmaz, ama üç dilin aynı girdileri kabul etmesi için aynı ön koşul korunur. Python'un yerleşik `pow(a, b, m)` fonksiyonu modüler üssü zaten hesaplar; burada yalnızca kâhindir. C'deki `pow()` ise `double` döndürür ve mod almaz.

9 · Test. C ve C++'ta kâhin, $b \le 10^6$ satırlarında saf O(b) çarpmadır; Python'da algoritma yine elle yazılır, yerleşik `pow(a, b, m)` **yalnızca kâhindir** ("kâhin" sütunu). Sınır durumları: b = 0, m = 1, en büyük çarpım ($a = m-1$, $b = 2^{64}-1$), ön koşul ihlali ($m \ge 2^{32}$); büyük b satırları Fermat'nın küçük teoremiyle (p asalsa $2^{(p-1)} \equiv 1 \pmod p$) de denetlenir.

ÇIKTI (ÜÇ DİLDE AYNI; SEÇİLİ SATIRLAR)

```
$ ./H02_11_hizli_us
          a                    b           m       sonuç  yin   öz      saf  kâhin
          7                    0          13           1    0    0        0  =
  123456789              1000000  1000000007   471040903   27   27  1000000  =
          3  1000000000000000000  1000000007   246336683   84   84        -  -
 4294967290  18446744073709551615  4294967291  4294967290  128  128        -  -
          2                   10  4294967311  ön koşul ihlali: m >= 2^32 (çarpım
taşabilir)
```

10 · Deney. Süre ölçmeye gerek yok: $b = 10^{18}$ için 84 çarpma, saf yöntemle 10^{18}. "yin" ve "öz" sütunlarındaki sayaçlar iki sürümde aynıdır ve formülle birebir örtüşür. **11 · Varyantlar.** Özyinelemeli sürüm (`us_mod_oz`, Ek A); $m \ge 2^{32}$ için taşmasız "ikiye katla ve ekle" çarpması (Hata Avcısı 7, düzeltilmiş); matris üssüyle Fibonacci sayıları (H13); RSA'nın temel işlemi (İleri Bakış).

4.4 En büyük alt dizi toplamı: $O(n^3) \to O(n^2) \to O(n)$ (Kadane)

1 · Problem. Toplamı en büyük **bitişik** alt diziyi (a[i..j]) bulmak. Sözleşme: n ≥ 1, alt dizi **boş olamaz** (tamamı negatif dizide yanıt en büyük eleman), eşitlikte ilk bulunan aralık. Bentley'nin (2000) dizisi {31, −41, 59, 26, −53, 58, 97, −93, −23, 84} için yanıt 187, a[2..6]. **2 · Fikir.** Üç büyüme sınıfı: **kübik** sürüm her (i, j) için a[i..j]'yi baştan toplar; **karesel** sürüm a[i..j] toplamını a[i..j−1] toplamına a[j] ekleyerek bulur; **Kadane** yalnızca "a[i]'de biten en iyi alt dizinin toplamını" (şimdiki) taşır: ya önceki alt diziyi uzatır ya a[i]'den yeniden başlar, hangisi büyükse. **3 · Elle izleme.** Şekil 2.12: i = 2'de uzat = −10 + 59 = 49 < 59, a[2]'den yeniden başlanır; i = 6'da şimdiki 187'ye ulaşır.

SÖZDE KOD · 4 · Sözde kod: Kadane

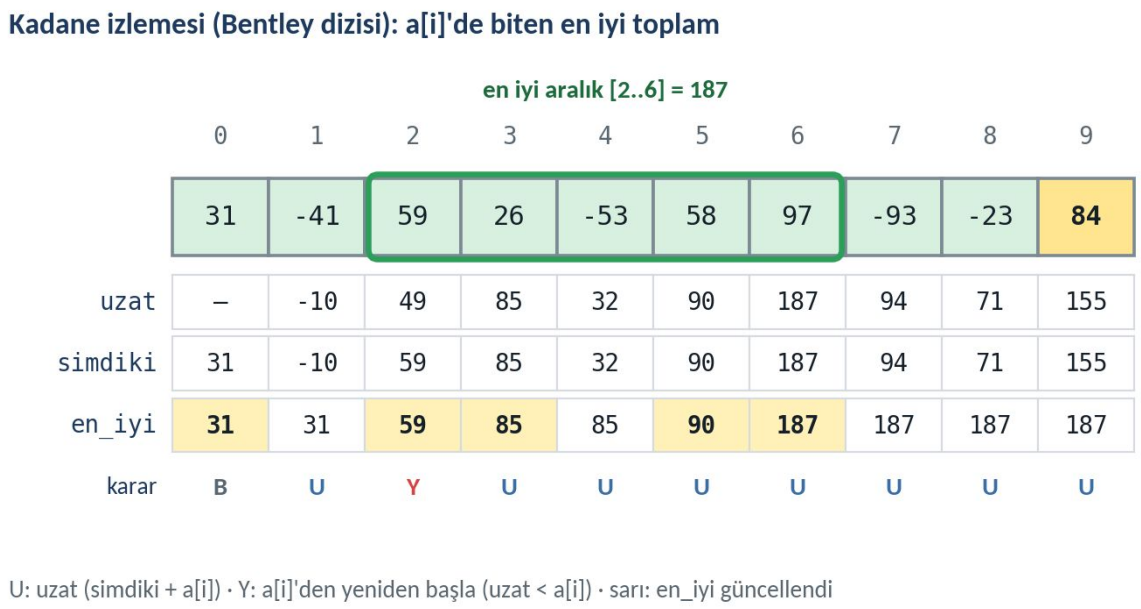
```
algoritma KADANE(a[0..n−1])              ▷ n ≥ 1; alt dizi boş olamaz
    şimdiki ← a[0];  en_iyi ← a[0]
    i ← 1'den n−1'e kadar her i için
        uzat ← şimdiki + a[i]
        eğer uzat < a[i] ise şimdiki ← a[i]     ▷ yeniden başla
        değilse şimdiki ← uzat                  ▷ uzat
        eğer şimdiki > en_iyi ise en_iyi ← şimdiki
    döndür en_iyi
```

5 · Doğruluk. Değişmez: *i. adımdan sonra şimdiki, a[i]'de biten en büyük alt dizi toplamıdır;* `en_iyi` *ise a[0..i]'deki en büyük alt dizi toplamıdır.* Başlatma (i = 0): tek alt dizi a[0..0]. Koruma: a[i]'de biten bir alt dizi ya yalnızca a[i]'dir ya da a[i−1]'de biten bir alt dizinin uzatılmışıdır; ikincilerin en büyüğü önceki şimdiki + a[i]'dir. Sonlanma: i = n − 1'de en_iyi yanıttır. **6 · Analiz.** Toplama sayıları kapalı biçimdir ve Örnek 13'ün sayaçlarıyla birebir aynıdır: kübik n(n + 1)(n + 2)/6, karesel n(n + 1)/2, Kadane n − 1; n = 10^5'te 166 671 666 700 000, 5 000 050 000 ve 99 999. Kadane her durumda Θ(n), ek bellek O(1).

Tablo 2.8
Kadane: algoritma kimlik kartı

ALGORİTMA KİMLİK KARTI **Kadane (en büyük alt dizi toplamı)** *(Kadane's algorithm, maximum subarray sum)*	
Strateji	Tek geçiş, artımlı güncelleme (H13: DP)
Girdi önkoşulu	n ≥ 1; alt dizi boş olamaz
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(1)
Özellikler	n − 1 toplama; aralığı (bas, son) da verir; tamamı negatif dizide en büyük elemanı döndürür. $O(n^3)$ ve $O(n^2)$ çözümlerle aynı sonucu verir (Bentley, 2000).
Değişmez / doğruluk	i. adımdan sonra `simdiki`, a[i]'de biten en büyük alt dizi toplamıdır; `en_iyi`, a[0..i] içindeki en büyük alt dizi toplamıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Tek geçişte en iyi bitişik aralık (ör. en kârlı dönem); akış verisi; 2B en büyük alt matris probleminin iç adımı
Ne zaman kullanılmaz	Bitişik olmayan seçimler (altküme problemi); boş alt diziye izin veren sözleşmede başlangıç değerleri 0 olmalıdır

Şekil 2.12
Kadane izlemesi (Bentley dizisi, programla üretildi): uzat, şimdiki ve en_iyi; U uzat, Y yeniden başla, sarı: en_iyi güncellendi



7 • C. Fonksiyon toplamın yanında aralığı (`bas`, `son`) ve toplama sayısını bir `Sonuc` yapısında döndürür; `s_bas` yeniden başlama noktasını tutar. **8 • Üç dil** (uzat ya da yeniden başla).

C **H02_13_en_buyuk_alt_dizi.c · satır 71–92**

```c
Sonuc kadane(const int *a, size_t n)
{
    Sonuc s = {a[0], 0, 0, 0};
    long long simdiki = a[0];
    size_t s_bas = 0;
    for (size_t i = 1; i < n; i++) {
        long long uzat = simdiki + a[i];
        s.topl++;
        if (uzat < a[i]) {
            simdiki = a[i];
            s_bas = i;
        } else {
            simdiki = uzat;
        }
        if (simdiki > s.toplam) {
            s.toplam = simdiki;
            s.bas = s_bas;
            s.son = i;
        }
    }
    return s;
}
```

C++ H02_13_en_buyuk_alt_dizi.cpp · satır 78–85

```cpp
        long long uzat = simdiki + a[i];
        s.topl++;
        if (uzat < a[i]) {
            simdiki = a[i];
            s_bas = i;
        } else {
            simdiki = uzat;
        }
```

Python H02_13_en_buyuk_alt_dizi.py · satır 54–59

```python
        uzat = simdiki + a[i]
        topl += 1
        if uzat < a[i]:
            simdiki, s_bas = a[i], i
        else:
            simdiki = uzat
```

DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**

C ve C++ dört sonucu (toplam, başlangıç, bitiş, toplama sayısı) bir yapıda (`Sonuc`) döndürür; Python bir demet döndürür ve `en_iyi, bas, son = toplam, i, j` çoklu atamasıyla günceller. C++'ta küçük diziler `{...}` başlatıcı listesiyle doğrudan `std::vector` parametresine verilir; Python'da fonksiyonlar bir listede tutulup (`surumler`) çağrılabilir.

9 · Test. Kübik sürüm kaba kuvvettir ve **kâhindir**: her dizide üç sürüm aynı toplamı ve aralığı vermelidir (rastgele dizilerde uyuşmazlık "!" ile işaretlenirdi). Sınır durumları: tümü negatif, tek eleman, sıfırlar, eşitlikte ilk aralık.

ÇIKTI (ÜÇ DİLDE AYNI)

```
$ ./H02_13_en_buyuk_alt_dizi
Bentley (n=10): kübik 187 [2..6] · karesel 187 [2..6] · Kadane 187 [2..6]
Tümü negatif (n=3): kübik -1 [1..1] · karesel -1 [1..1] · Kadane -1 [1..1]
Tek eleman (n=1): kübik 5 [0..0] · karesel 5 [0..0] · Kadane 5 [0..0]
```

10 · Deney (Kadane yarışması, Bölüm 11.4). Bu makinede C'de $n = 10^3$ için kübik 76,8 ms, karesel 0,31 ms, Kadane 0,001 ms; $n = 10^5$'te karesel 3,26 s, Kadane 0,111 ms. Kübik sürüm $n = 4000$'de 5,06 s sürdü; oran testiyle $(\times (10/4)^3)$ $n = 10^4$ için ≈ 79 s, $n = 10^5$ için ≈ 21,9 saat **kestirilir** (ölçülmedi). **11 · Varyantlar.** Boş alt diziye izin veren sözleşmede şimdiki ve en_iyi 0'dan başlar (Bentley, 2000); en büyük alt matris, satır çiftleri üzerinde Kadane'yi iç adım olarak kullanır; aynı problem H6'da böl ve fethetle O(n log n), H13'te dinamik programlamayla yeniden çözülecek.

5. Çözümlü Örnekler: Merdivenin Kalan Basamakları

Bölüm 3 ve 4'te Örnek 1–4, 6–9, 11 ve 13'ü kullandık. Kalan basamaklar aynı araçları yeni bir bağlamda dener. Bütün örneklerin C++ ve Python karşılıkları Ek A'dadır.

5.1 Örnek 5 ★★ — Yinelenen öğrenci numarası var mı?

İki sürüm aynı bağımlı iç içe döngüyü kullanır: `yinelenen_var_mi` ilk eşit çiftte **erken çıkar**, `yinelenen_bul` bütün çiftleri tarar. Öğrenci numaraları 2 400 000 000'dan başlar ve 32 bitlik `int`'e sığmaz (INT_MAX = 2 147 483 647): `long long` gerekir.

H02_05_yinelenen_numara.c · satır 19–35

```c
/* İlk eşit çifti (i, j) bulunca durur; *kar: karşılaştırma sayısı */
bool yinelenen_var_mi(const long long *no, size_t n, size_t *ilk_i,
                      size_t *ilk_j, long long *kar)
{
    *kar = 0;
    for (size_t i = 0; i + 1 < n; i++) {
        for (size_t j = i + 1; j < n; j++) {
            (*kar)++;
            if (no[i] == no[j]) {
                *ilk_i = i;
                *ilk_j = j;
                return true;                    /* erken çıkış */
            }
        }
    }
    return false;
}
```

Programın ilk dört satırı Mini-OBS 2 v2.1'in test senaryoları 1–4'tür (ör. S2, n = 6: erken çıkış 2 karşılaştırmada (0, 2) çiftini bulur, tüm çiftler 15 karşılaştırmada 4 çift). İkinci tablo 500, 1000 ve 2000 numaralık dosyalarda iki durumu karşılaştırır: temiz dosyada (yinelenen yok, **en kötü durum**) iki sürüm de tam n(n − 1)/2 karşılaştırma yapar; tek bir yinelenen eklenince erken çıkış 454, 479, 1724 karşılaştırmada durur. Mini-OBS'nin olağan kullanımında (temiz dosya) en kötü durum **tipik** durumdur: erken çıkışın kazancı yoktur. Sarmal: H6'da sıralayıp komşuları karşılaştırarak O(n log n), H10'da karma tablosuyla ortalama O(n).

5.2 Örnek 6 ★★ — Büyüme tablosu üreteci ve taşma

2^n ve n! değerleri `double`'ın aralığını (≈ $1{,}8 \cdot 10^{308}$) hızla aşar. Diller burada ayrışır: C/C++'ta `pow(2.0, 10000.0)` `inf` döndürür, Python'da `2.0 ** 10000` `OverflowError` fırlatır. Örnek, büyüklüğü **logaritmayla** hesaplar ve sığmayan değeri 10^k biçiminde yazar; böylece üç dilin çıktısı bayt bayt aynı kalır:

H02_06_buyume_tablosu.c · satır 30–40

```c
void iki_us_yaz(long n)
{
    double lg = (double)n * log10(2.0);
    double d = 1.0;
    if (lg < 308.0) {
        for (long i = 0; i < n; i++) {
            d *= 2.0;
        }
    }
    deger_yaz(d, lg);
}
```

Örnek, “1 saniyede en büyük n” değerlerini (Tablo 2.4) n log n için ikiye katlayıp ikili arayarak, n^2 ve n^3 için tamsayı sayarak bulur; saniyedeki işlem sayısı komut satırından verilebilir (`./H02_06_buyume_tablosu 1000000000`).

5.3 Örnek 10 ★★★ — Öklid’in en kötü durumu: Fibonacci (Lamé)

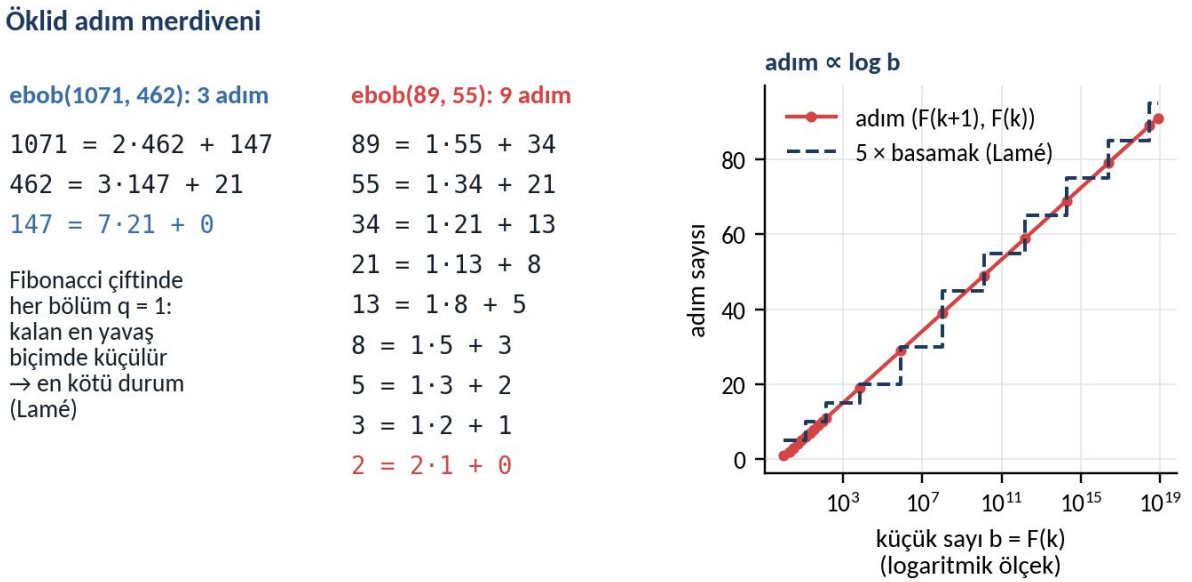
Öklid algoritması (Knuth’un, 1997, algoritma kavramını tanıttığı ilk örnek) ebob(a, b) = ebob(b, a mod b) özdeşliğini b = 0 olana kadar uygular. Her iki adımda küçük sayı en az yarıya iner; adım sayısı O(log min(a, b))’dir. En kötü durum, her bölümün 1 olduğu, yani kalanın olabilecek en yavaş biçimde küçüldüğü **ardışık Fibonacci sayılarıdır** (ebob(1071, 462) 3 adımda, ebob(89, 55) 9 adımda biter). Lamé’nin sonucuna göre adım sayısı küçük sayının ondalık basamak sayısının beş katını aşmaz (Cormen vd., 2022).

C `H02_10_oklid_lame.c · satır 18–28`

```c
uint64_t ebob(uint64_t a, uint64_t b, int *adim)
{
    *adim = 0;
    while (b != 0) {
        uint64_t r = a % b;
        a = b;
        b = r;
        (*adim)++;
    }
    return a;
}
```

Program ayrıca 1 ≤ b < a ≤ 500 aralığındaki 124 750 çiftin **hepsini** dener: s adım gerektiren en küçük çift her s için tam olarak (F(s+2), F(s+1)) çıkar. Bu, iddianın kaba kuvvetle sınanmasıdır; kanıtı Algoritma Analizi dersine kalır. F(93) < 2^{64} < F(94) olduğundan C tablosu k = 92’de durur (Şekil 2.13).

Şekil 2.13
Öklid’in adım merdiveni: ebob(1071, 462) 3 adımda biter; ardışık Fibonacci sayılarında her bölüm 1’dir ve adım sayısı en yüksektir (ebob(89, 55): 9 adım). Sağda adım sayısı log b ile doğrusal büyür ve Lamé sınırının altında kalır



5.4 Örnek 12 ★★★ — Büyük tamsayılar ve sabit zamanın sınırı (Δ)

Aynı `iki_us` döngüsü üç dilde çalışır ama sonuçlar ayrışır: C ve C++'ta `uint64_t` **işaretsizdir** ve taşma tanımlıdır: değer 2^{64} modunda sarar (ISO/IEC, 2018); n ≥ 64 için sonuç 0 olur. Python `int`'i kesin değeri tutar; bedeli Kaputun Altında 2'de ölçtüğümüz süredir. Bu yüzden örnek "Δ" türündedir: çıktılar bilinçli olarak farklıdır.

C `H02_12_buyuk_tamsayi.c · satır 20–27`

```c
uint64_t iki_us(int n)
{
    uint64_t x = 1;
    for (int i = 0; i < n; i++) {
        x = x * 2;
    }
    return x;
}
```

İki dilin çıktıları Ek A.12'de yan yanadır. Ortak kalan son satırlar da öğreticidir: 3^{39} tamsayı çarpımıyla 4 052 555 153 018 976 267, `pow(3.0, 39.0)` ile 4 052 555 153 018 976 256'dır (fark 11): `double` yalnızca 53 bitlik anlamlı kısım tutar. Python 3.11'de (ve güvenlik güncellemeli 3.10 sürümlerinde) 4300 basamaktan uzun bir tamsayıyı `str()` ile yazdırmak hata verdiği için örnek bit uzunluğunu, son altı basamağı ve $10^9 + 7$ kalanını yazar (Python Software Foundation, t.y.).

6. Haftanın Deneyi: Üç Gizemli Fonksiyonu Sınıflandırmak

Soru. İç yapısı gizli üç fonksiyonun (gizemli_1, gizemli_2, gizemli_3) büyüme sınıfını **yalnızca süre ölçümlerinden** bulabilir miyiz? Biri doğrusal, biri n log n, biri karesel. **Düzen.** `kodlar/deney/` altındaki sürücü (`oran_deneyi.c`, `.cpp`, `.py`) diziyi ortak LCG ile üretir (tohum 42), n'i ikiye katlar, her n için 1 ısınma + 5 tekrar yapar ve **ortancayı** CSV'ye yazar; en çok–en az yayılımı ortancanın %30'unu aşan ölçüm en çok iki kez yinelenir. Dört yapılandırma: C `-O2`, C `-O0`, C++ `-O2`, Python 3.11. Fonksiyonlar diziyi yalnızca okur ve bir sağlama değeri döndürür; üç dilde her n için sağlama değerleri aynıdır (aynı algoritma, aynı veri).

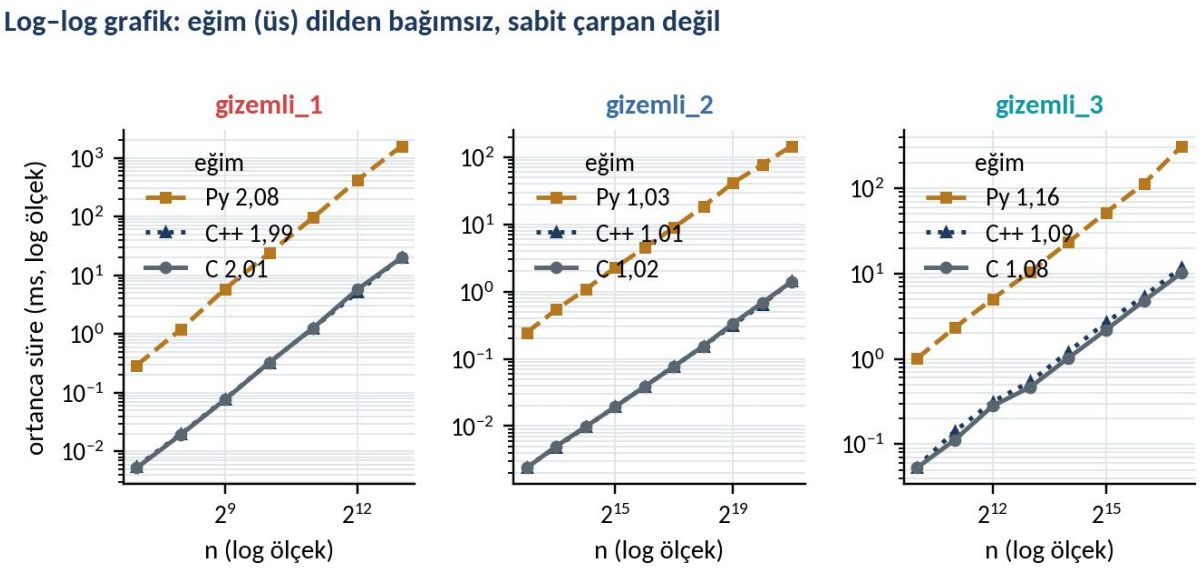
C `deney/c/gizemli.h · satır 14–16`

```c
long long gizemli_1(const int *a, size_t n);
long long gizemli_2(const int *a, size_t n);
long long gizemli_3(const int *a, size_t n);
```

KOMUTLAR

```
$ gcc -std=c17 -Wall -Wextra -pedantic -O2 -I../../c oran_deneyi.c gizemli.c \
      ../../c/ortak_rastgele.c ../../c/zamanlayici.c -o oran_deneyi
$ ./oran_deneyi 1 128 8192 5 > g1.csv      # FONK N_BAS N_SON TEKRAR
```

Şekil 2.14
Log–log grafik: üç kara kutu, C / C++ / Python; doğrular paralel (eğim dilden bağımsız), dikey kayma sabit çarpandır (bu makinede)

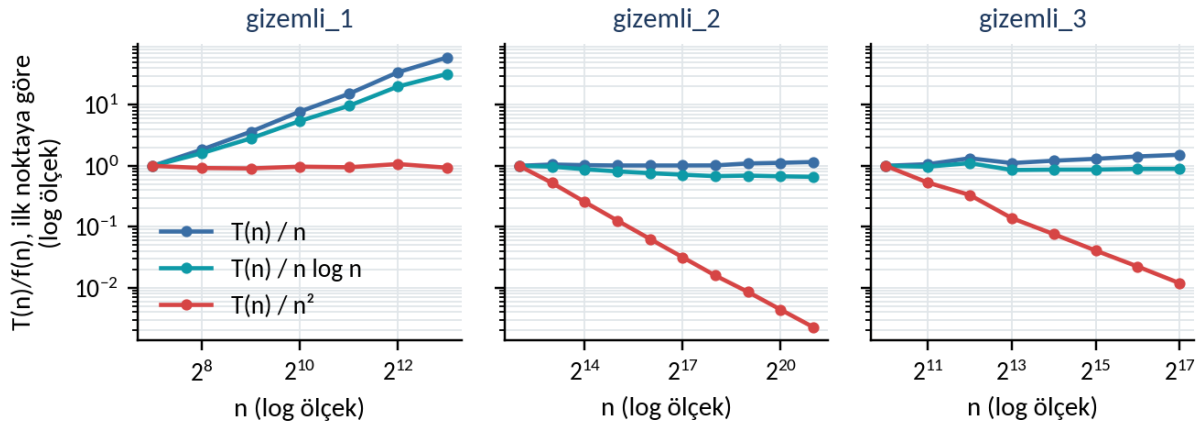


Tablo 2.9
Haftanın Deneyi sonuçları (bu makinede; deney/sonuclar/.csv): oranlar, eğim, birim iş başına süre ve T(n)/f(n) düzlük testinin seçtiği model*

Fonksiyon	Yapılandırma	n (ortak ızgara)	Oran (üst yarı ortancası)	Log–log eğim	Sabit (son n)	Düzlük testi
gizemli_1	C -O2	128–8 192	3,91	2,01	0,60 ns/çift	n^2
gizemli_1	C++ -O2	128–8 192	3,94	1,99	0,60 ns/çift	n^2
gizemli_1	C -O0	128–8 192	4,16	2,09	4,76 ns/çift	n^2
gizemli_1	Python	128–8 192	4,05	2,08	47,5 ns/çift	n^2
gizemli_2	C -O2	4 096–2 097 152	2,06	1,02	0,67 ns/eleman	n
gizemli_2	C++ -O2	4 096–2 097 152	2,05	1,01	0,68 ns/eleman	n
gizemli_2	C -O0	4 096–2 097 152	2,04	1,06	6,83 ns/eleman	n log n
gizemli_2	Python	4 096–2 097 152	1,98	1,03	70,2 ns/eleman	n
gizemli_3	C -O2	1 024–131 072	2,15	1,08	4,60 ns/eleman·$\log_2 n$	n log n
gizemli_3	C++ -O2	1 024–131 072	2,19	1,09	5,32 ns/eleman·$\log_2 n$	n log n
gizemli_3	C -O0	1 024–131 072	2,15	1,13	8,57 ns/eleman·$\log_2 n$	n log n
gizemli_3	Python	1 024–131 072	2,24	1,16	141,2 ns/eleman·$\log_2 n$	n log n

Sınıflandırma. (1) **Oran testi:** gizemli_1'in katlama oranları ≈ 4'tür (C -O2 ortancası 3,91): $\Theta(n^2)$. gizemli_2 ve gizemli_3'te oran ≈ 2'dir (2,06 ve 2,15): ya $\Theta(n)$ ya $\Theta(n \log n)$. (2) **Log–log eğim:** 2,01, 1,02 ve 1,08; n log n'nin eğimi bu aralıkta 1'den biraz büyüktür. (3) **Düzlük testi:** T(n)/f(n) doğru f için düz kalır; değişim katsayısı en küçük olan model seçilir (Şekil 2.15). Bu üç adım C -O2, C++ ve Python'da üç fonksiyonu da doğru sınıflandırdı (gerçek sınıflar: n^2, n, n log n).

Şekil 2.15
Düzlük testi (C -O2, ortak ızgara, bu makinede): T(n)/f(n) ilk noktaya göre normalize edildi; doğru f için eğri düz kalır (gizemli_1: n^2, gizemli_2: n, gizemli_3: n log n)



Eğim aynı, sabit farklı. Birim iş başına süre Python'da C -O2'ye göre gizemli_1'de ×79,6, gizemli_2'de ×104,3, gizemli_3'te ×30,7; C++ ≈ C. Ama doğrular paraleldir: dil sınıfı değiştirmez, doğruyu yukarı kaydırır. **Sınırlılık.** C -O0'da gizemli_2 yanlışlıkla n log n sınıflandı (tabloda vurgulu satır): ilk iki katlamanın oranları 2,36 ve 2,23 çıktı (on mikrosaniyeler süren kısa ölçümlerde sabit maliyet ve gürültü), büyük n'de de dizi L2 önbelleğini aştı. log n çarpanı bir katlamada yalnızca (1 + 1/log n) kadar fark yaratır, n = 2^{16}'da 2,13'e karşı 2,00; önbellek etkisi de bu kadar fark yaratabilir. **Saat çözünürlüğü:** bu ölçümlerde C sürücüsünün saati ≈ 0,24 µs çözünürlüklüydü (`zaman_sn()` 1970'ten bu yana geçen saniyeyi `double` içinde tutuyordu; kod arşivindeki `zamanlayici` ilk çağrıya göreli zaman kullanır ve nanosaniye düzeyindedir). En küçük n'lerde birkaç mikrosaniye süren C -O2 ölçümleri bu yüzden %5–10'a kadar nicemlidir; ±1 adımlık nicemleme eğimi en çok ±0,005 değiştirir ve hiçbir sınıflandırmayı değiştirmez. Ölçümler 2 çekirdekli, başka süreçlerin de çalıştığı bir sanal makinede yapıldı (makine bilgisi `deney/OLCUM_ORTAMI.txt`); deneyi kendi makinenizde tekrarlayın (Bölüm 11.3).

7. Hata Avcısı: Gizli Maliyetler ve Yanlış Sonuçlar

Aşağıdaki sekiz hatanın hiçbiri derleyici uyarısı üretmez (`-Wall -Wextra -pedantic` ile 0 uyarı); hepsini çalıştırarak ya da ölçerek yakalarız. Belirtiler bu makinedeki gerçek çıktılardır (`kodlar/hata_avcisi/ciktilar/`).

Tablo 2.10
Hata Avcısı: hata | belirti | neden | çözüm (dosyalar kodlar/hata_avcisi/)

Hata	Belirti (bu makinede)	Neden	Çözüm
HA1 · "O(n) her zaman $O(n^2)$'den hızlıdır"	16 numaralık 20 000 grupta "O(n)" işaret dizisi 546,6 ms; melez yöntem 1,8 ms	"O(n)" yöntemin gerçek maliyeti O(n + ARALIK): her çağrıda 1 MB sıfırlama; küçük n'de sabit baskın	Sabitleri ölçmek; kesişim noktasını bulup melez yöntem (n < eşik için kaba kuvvet)
HA2 · O'yu "en kötü durum" sanmak; en iyi durumla ölçüp genellemek	aranan = a[0]: oran ≈ 1,00 → rapor "doğrusal arama Θ(1)"	Ölçülen yalnızca en iyi durum (1 karşılaştırma)	Girdi ailelerini ayrı ölçmek; raporda durum + gösterim (en kötü Θ(n), her durumda O(n))
HA3 · Koşulda gizli maliyet: `i < strlen(s)`	n = 160 000'de 284,0 ms; oranlar ≈ 4	`strlen` O(n) ve her turda çağrılır; gövde s'yi değiştirdiği için derleyici döngü dışına alamaz	Uzunluğu bir kez hesaplamak: aynı n'de 0,208 ms

Hata	Belirti (bu makinede)	Neden	Çözüm
HA4 · Python'da `x in liste`	tekilleştirme n = 32 000'de 3,9 s; oranlar ≈ 3,4–3,8	Liste üyeliği doğrusal tarama; döngü içinde $\Theta(n^2)$	Görülenleri `set`'te tutmak (ortalama O(1)): 6,1 ms
HA5 · Kalburda `i * i` taşması (`int`)	N = 46 340'a kadar doğru; N = 100 000'de SIGSEGV (çıkış 139); UBSan: signed integer overflow	$46\,349^2$ > INT_MAX: işaretli taşma **tanımsız davranıştır**; burada negatif indisle dizi dışına yazıldı	Dış döngüde `i <= n / i`, iç sayaç `long long`
HA6 · İç döngüyü `2 * i`'den başlatmak, dış döngüyü n'ye kadar sürdürmek	π(N) aynı; N = 10^6'da işaretleme 2 775 210 ↔ 2 122 048	i · i'den küçük katlar zaten işaretli; doğruluk değil verimlilik hatası	`i * i <= n` ve `j = i * i`
HA7 · `a * b % m` çarpımının 64 biti aşması	m = 10^9 + 7'de doğru; m = 999 999 999 989'da Fermat testi 2^(m−1) mod m = 0 (doğrusu 1)	m ≈ 10^{12} ⇒ çarpım ≈ 10^{24} > 2^{64}; `uint64_t` sessizce sarar	m < 2^{32} ön koşulunu denetlemek ya da taşmasız `carp_mod`
HA8 · Sonucu kullanılmayan işlemi ölçmek	`-O2`: n 16 kat büyüdü, süre ≈ 0 ms → "O(1)?!"; `-O0`'da oran ≈ 4	Yan etkisiz çağrının sonucu atılınca derleyici çağrıyı ölü kod olarak siler	Sonucu kullanmak (toplayıp yazdırmak); süre sıfıra yakınsa önce ölçümden şüphelenmek

HA3 · Döngü koşulunda gizli maliyet (GTA Online'daki desen)

C **H02_HA3_strlen_kosulda.c · satır 25–35**

```c
size_t ayirici_degistir(char *s)
{
    size_t degisen = 0;
    for (size_t i = 0; i < strlen(s); i++) {    /* HATA: her turda O(n) */
        if (s[i] == ',') {
            s[i] = ';';
            degisen++;
        }
    }
    return degisen;
}
```

Belirtiyi Tablo 2.10 gösterir (oranlar ≈ 4). Düzeltme tek satırdır: `size_t uzunluk = strlen(s);` döngüden önce hesaplanır (`H02_HA3_strlen_kosulda_duzeltilmis.c`, satır 27–28); oranlar ≈ 2'ye iner. C++'ta `s.size()` ve Python'da `len(s)` uzunluğu nesnede sakladığı için bu tuzak yoktur (Dil Penceresi).

HA5 · Kalburda int taşması: tanımsız davranış

C **H02_HA5_kalbur_tasma.c · satır 23–29**

```c
    for (int i = 2; i <= n; i++) {              /* HATA 1: i * i <= n değil */
        if (!bilesik[i]) {
            for (int j = i * i; j <= n; j += i) {   /* HATA 2: i*i taşar */
                bilesik[j] = 1;
            }
        }
    }
```

`-fsanitize=undefined` ile derlenen sürüm N = 46 340'a kadar doğru sayar (4 792 asal), N = 100 000'de UBSan'ın "signed integer overflow: 46349 * 46349 cannot be represented in type 'int'" iletisini yazıp SIGSEGV ile sonlanır. İşaretli taşma C'de **tanımsız davranıştır** (ISO/IEC, 2018): program sarabilir, çökebilir ya da sessizce yanlış sonuç verebilir. Bu makinede `-O0` ve `-O2` derlemeleri SIGSEGV ile sonlandı; "her zaman çöker" demek yanlıştır. Düzeltilmiş sürüm dış döngüyü `i <= n / i` ile (taşmasız) yazar ve iç döngü sayacını `long long` yapar; N = 10^7 için 664 579 asal bulur.

HA7. Küçük m'li testler ($10^9 + 7$, $2^{32} - 5$) geçtiği için hata gözden kaçar: kâhinle karşılaştırırken **girdi uzayının sınırlarını** da deneyin. Düzeltme ya $m < 2^{32}$ ön koşulunu denetler (Örnek 11) ya da taşmasız "ikiye katla ve ekle" çarpması kullanır (`H02_HA7_modmul_tasma_duzeltilmis.c`, `carp_mod`).
HA8. Sonucu atılan `iki_toplam(a, n);` çağrısını `-O2` siler; `-O0` derlemesinde oranlar ≈ 4'tür (`hata_avcisi/ciktilar/H02_HA8_olu_kod_olcumu_O0.txt`). Bütün hatalı ve düzeltilmiş sürümler `kodlar/hata_avcisi/` klasöründedir; sunumda ve web sayfasında yan yana inceleniyor.

8. Dil Penceresi: Aynı Görünen Kod, Farklı Maliyet

Bu haftanın dil farkları sözdiziminde değil **maliyette** ve **sayısal davranıştadır** (Tablo 2.11).

Tablo 2.11
Dil Penceresi: C, C++ ve Python'da bu haftanın maliyet ve sayı farkları

Öğe	C	C++	Python
Dizgi uzunluğu	`strlen(s)`: O(n), her çağrıda '\0' aranır	`s.size()`: O(1), nesnede saklı	`len(s)`: O(1), nesnede saklı
Üyelik sınaması	elle doğrusal tarama: O(n)	`std::find` O(n); `std::unordered_set` ort. O(1)	`x in liste` O(n); `x in kume` ort. O(1)
Sabit genişlikli tamsayı	`uint64_t` mod 2^{64} sarar (tanımlı); işaretli taşma tanımsız davranış	C ile aynı	`int` sınırsız; maliyet basamak sayısıyla büyür
Üs alma	`pow()`: double, 53 bit (3^{39}'da fark 11)	`std::pow`: double	`pow(a, b, m)`: kesin, modüler; `a ** b` kesin
Kayan nokta taşması	`pow(2.0, 10000.0)` → inf	`std::pow` → inf	`2.0 ** 10000` → `OverflowError`
Büyük sayıyı yazdırma	`printf` ile `%llu` (en çok 20 basamak)	`std::cout << x`	`str()`: 3.11'de 4300 basamak sınırı (`ValueError`)
Türkçe harfle dolgu	`%-5s` bayt sayar (ı = 2 bayt)	`std::setw` bayt sayar	`f"{s:<5}"` karakter sayar

İlk satırı ölçelim: aynı "boşlukları alt çizgiye çevir" döngüsü, koşulu her turda yeniden hesaplayan bir `while` ile üç dilde yazıldı (`kodlar/kavram/H02_K04_uzunluk_maliyeti.c`, `.cpp`, `.py`; C'de koşul HA3'teki gibi `i < strlen(s)`, C++'ta `i < s.size()`, Python'da `i < len(s)`).

Tablo 2.12
Koşulda uzunluk: C'de oran ≈ 4 (Θ(n²)), C++ ve Python'da ≈ 2 (Θ(n)); bu makinede tek ölçüm

n	C süre (ms) · oran	C++ süre (ms) · oran	Python süre (ms) · oran
10 000	1,656 · —	0,007 · —	1,532 · —
20 000	5,936 · 3,59	0,012 · 1,83	2,893 · 1,89
40 000	30,784 · 5,19	0,024 · 1,99	6,180 · 2,14
80 000	103,799 · 3,37	0,047 · 1,99	11,903 · 1,93

Burada sınıf farkı dil farkını yener: n = 10 000'de bile C'nin karesel sürümü Python'un doğrusal sürümünden yavaştır ve n büyüdükçe fark açılır (n = 80 000'de ≈ 9 kat). Ölçümler tek çalıştırmadır; C'deki oranların dalgalanması (3,4–5,2) gürültüdür, sınıf oranların gidişinden okunur.

Kütüphane karşılıkları bu haftanın algoritmalarının çoğu için yoktur (asallık testi ve kalbur hiçbir dilin standart kütüphanesinde bulunmaz); olanlar: C++17 `std::gcd`, Python `math.gcd` (Öklid), Python `pow(a, b, m)` (hızlı üs; bu derste yalnızca kâhin), Python `int.bit_length()` ($\lfloor \log_2 n \rfloor + 1$), C++ `std::accumulate` ve Python `sum` (doğrusal toplam). Her örnek dosyası bu notla biter.

Tablonun son satırı küçük ama gerçek bir tuzaktır: UTF-8'de "ı" iki bayttır; C'de `printf("%-5s", "hayır")` genişliği **bayt** olarak sayar (6 bayt, dolgu yok), Python **karakter** sayar. Bu yüzden Örnek 8 "evet " ve "hayır" dizgilerini sabit yazar: üç dilin çıktısı ancak böyle bayt bayt aynı olur.

9. Mühendis Gözüyle: Karmaşıklık Bütçesi ve Gizli Karesel Kod

Girdi boyutundan algoritma sınıfına geri çıkarım. Bir problemin üst sınırı ve süre hedefi biliniyorsa, uygulanabilirlik tablosu (Tablo 2.4) hangi sınıfın kabul edilebilir olduğunu baştan söyler. Saniyede ≈ 10^8 basit işlem ve 1 saniyelik bütçeyle kaba kural: $n \le 11 \to n!$ bile olur; $n \le 26 \to 2^n$; $n \le 464 \to n^3$; $n \le 10^4 \to n^2$; $n \le 4{,}5 \cdot 10^6 \to n \log n$; daha büyük n → n ya da daha iyisi. Mini-OBS 2'nin hedefi olan 10^6 kayıtta $\Theta(n^2)$ yinelenen denetimi $n(n-1)/2 \approx 5 \cdot 10^{11}$ karşılaştırma yapar: kaba kurala göre ≈ 5000 s, yani bütçenin ≈ 5000 katı. Bu makinede bir karşılaştırma ≈ 0,7 ns sürdüğü için v2.1 ölçümüyle kestirim ≈ 340 s'dir (≈ 5,7 dakika): kural sabiti kötümser tahmin etse de bütçe yine yüzlerce kat aşılır. Bu yüzden H6 ve H10'da daha iyi algoritmalar yazacağız.

- **Gizli karesel kod üretimde pahalıdır.** GTA Online'da (t0st, 2021) iki küçük desen oyunculara dakikalar kaybettirdi. Kod incelemesinde döngü içindeki her çağrının maliyetini sorun: `strlen`, `x in liste`, `list.index`, dizinin başına ekleme.
- **Ölçümü sayaçla destekleyin.** Süre gürültülüdür; işlem sayacı gürültüsüzdür ve kuramla birebir karşılaştırılabilir: işlem sayısı da bir sözleşmedir (Mini-OBS 2 v2.1'in kabul ölçütleri).
- **Sabit çarpan kırılgandır, büyüme sınıfı değildir.** Derleme seçeneği, önbellek ya da dil sabiti birkaç kat değiştirir, eğimi değiştirmez (Bölüm 6). İki algoritmayı aynı programda, aynı derlemeyle karşılaştırın.
- **Ön koşulları kodda yazılı tutun.** $m < 2^{32}$ (hızlı üs), $n < 2^{63}$ (deneme bölmesi), N + 1 baytın belleğe sığması (kalbur): sessizce yanlış sonuç üreten kod, yavaş koddan daha tehlikelidir.

10. Mini-OBS 2 Adımı: v2.1 — Sayaçlar ve Yinelenen Numara

Bu hafta: v2.0'da yüklemenin karesel olduğunu ölçtük. Şimdi işlemleri sayarak T(n)'yi yazıyor, yinelenen numara tespitinin kaba kuvvet sürümünü kuruyor ve oran testiyle büyüme sınıfını deneysel olarak doğruluyoruz. Örnek 5 bu adımın sade (dizi tabanlı) bir provasıdır.

Gereksinimler

- `sayac.c`: `sayac.h`'deki `Sayac` yapısı, `SAYAC_EKLE` makrosu, `sayac_sifirla`, `sayac_yaz`.
- `obs.c`: `dizi_ortalama_say`, `standart_sapma_say`, `min_max_indis_say`, `ders_istatistigi_say`, `sinif_numara_bul_say`; eski adlar bunları `NULL` ile çağıran sarmalayıcılar olmalı. Her fonksiyonun başına T(n)'yi yorum olarak yazın.
- `yinelenen.c`: `yinelenen_var_mi` (erken çıkış; kayıt dizisinde), `yinelenen_bul` (tüm çiftler; çiftleri sözlük sırasında `azami` sınırıyla yazar, toplamı döndürür), `tum_ciftler`.
- `olcer.c`: `yinelenen` ve `erken` işlemleri; her n için sayım çalıştırması + sayaç kapalı süre ölçümü; CSV'de karşılaştırma sayısı.
- `denetle.c` (önerilen): bir CSV'deki bütün yinelenen numaraları sıra numaralarıyla listeleyen araç; yinelenen varsa çıkış kodu 1.
- `test_obs.c` ve `test_kahin.c`: sayaç formülleri ve yinelenen tespiti için birim ve kâhin testleri.
- Ölçüm raporu: n = 10^3, $2{\cdot}10^3$, …, ≥ $6{,}4{\cdot}10^4$ için tablo (n, karşılaştırma, süre, oran), log–log grafik, eğim ve yorum; makine bilgisi.

İpuçları

- Temel işlemi önce **yazıyla** seçin ("iki numaranın karşılaştırılması"); sonra yalnızca onu sayın. Döngü sayaçları ve dizi indeksleri sayılmaz.
- `for (j = i + 1; …)`: iç döngünün `i + 1`'den başlaması hem her çifti **bir kez** saymayı hem de kaydın kendisiyle karşılaştırılmamasını sağlar. `j = 0`'dan başlayan sürüm $n^2 - n$ karşılaştırma yapar ve her çifti iki kez bulur.
- `n(n − 1)/2`'yi `int` ile hesaplamayın: n = 10^5'te 4 999 950 000 değeri 32 bitlik `int`'i aşar; `unsigned long long` kullanın.

Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; bütün birim ve kâhin testleri geçer.
2. Sayaçlar kapalı biçimlerle **birebir** aynıdır: `dizi_ortalama_say` (n, 1), `standart_sapma_say` (3n, n + 2), `min_max_indis_say` 2(n − 1), `ders_istatistigi_say` (2n − 2, 4n, n + 3), `sinif_numara_bul_say` i + 1 ya da n. `./olcer istatistik --n 1000,2000,4000` her satırda `kuram: UYUMLU` yazar.
3. `yinelenen_bul`'un karşılaştırma sayısı her girdide tam n(n − 1)/2'dir (yinelenen olsun olmasın); `yinelenen_var_mi` yinelenen yokken n(n − 1)/2, ilk iki kayıt aynıyken 1 karşılaştırma yapar.
4. `./denetle testler/yinelenen_k3.csv`: 435 karşılaştırma, 3 çift (sıra 8 ↔ 11, 10 ↔ 29, 17 ↔ 30), çıkış kodu 1; `--erken`: 185 karşılaştırma, ilk çift 8 ↔ 11; `./denetle ornek_veri/ogrenciler.csv`: 28 karşılaştırma, 0 çift, çıkış 0.
5. Oran testi (kendi makinenizde, tüm çiftler, numara dizisi): n ≥ 4 000 için her katlama oranı **3,5 ile 4,5 arasında**; en az 7 katlamalı ölçümden hesaplanan log–log eğim **1,9 ile 2,1 arasında**. Ders istatistiği için eğim 0,8–1,3 (doğrusal).
6. Sayaç açık ve kapalı ölçümler arasındaki fark, tekrarların en az–en çok aralığı içinde kalır.
7. `--hata-goster yinelenen` ile kâhin hatalı sürümü yakalar (karşılaştırma sayısı ya da çift listesi farklı), 1 ile çıkar.
8. `make asan` temiz.

Tablo 2.13
Mini-OBS 2 v2.1 test senaryoları

#	Girdi	Beklenen çıktı / davranış
1	numaralar {11, 22, 33, 44, 55}	`yinelenen_bul` 0 çift, 10 karşılaştırma; `yinelenen_var_mi` false, 10
2	{7, 3, 7, 5, 3, 7}	4 çift: (0,2) (0,5) (1,4) (2,5); 15 karşılaştırma; erken: (0,2), 2 karşılaştırma
3	{9, 9, 1, 2, 3, 4}	erken: 1 karşılaştırma (en iyi durum)
4	{1, 2, 3, 4, 8, 8}	erken: (4,5), 5 + 4 + 3 + 2 + 1 = 15 karşılaştırma (son çift)
5	n = 0 ve n = 1	0 çift, 0 karşılaştırma
6	`azami = 2`, örnek 2	dönüş 4; yalnızca (0,2) ve (0,5) yazılır
7	a = {4, 1, 3, 1, 4}	`min_max_indis_say`: enk 1, enb 0 (ilk indisler), 8 karşılaştırma
8	`./denetle testler/yinelenen_k3.csv`	3 çift, 435 karşılaştırma, çıkış 1
9	`./denetle testler/bozuk.csv`	"dosya biçimi hatalı (satır 3)", çıkış 1
10	`./denetle`	kullanım iletisi, çıkış 2
11	`./olcer yinelenen --n 1000,2000,4000,8000`	sayaç 499 500 · 1 999 000 · 7 998 000 · 31 996 000, `kuram: UYUMLU`; stderr'de oran ≈ 4
12	`./test_kahin --hata-goster yinelenen`	"KÂHİN UYUŞMAZLIĞI", karşılaştırma farkı n − 1, çıkış 1

Başlangıç noktası. Verilir: kendi v2.0'ınız (ya da referans v2.0), `sayac.h`, `yinelenen.h` (bildirimler ve sözleşmeler), `testler/yinelenen_k3.csv`. Yazılacak ve değiştirilecek dosyaların listesi Mini-OBS 2 v2.1 adım belgesindedir.

11. Alıştırmalar ve Atölye

Alıştırmaların çözümleri öğrenci sürümünde yer almaz. Kod içeren alıştırmaların dosyaları `kodlar/alistirmalar/` klasöründedir; önce elle çözün, sonra programı çalıştırarak denetleyin. Dört atölye etkinliği 11.2–11.5'tedir.

11.1 Alıştırmalar (A–G)

A · İzleme ve çıktı tahmini

Alıştırma 2.1 ★☆☆ *Çıktı tahmini*

Aşağıdaki programı çalıştırmadan s1, s2 ve s3'ü bulun; sonra genel n için her birinin kapalı biçimini ve büyüme sınıfını yazın.

C **H02_A01_cikti_tahmini.c · satır 13–28**

```c
    int n = 16;
    int s1 = 0, s2 = 0, s3 = 0;
    for (int i = 1; i < n; i *= 2) {
        for (int j = 0; j < n; j++) {
            s1++;
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            s2++;
        }
    }
    for (int i = n; i > 0; i /= 4) {
        s3++;
    }
    printf("s1 = %d, s2 = %d, s3 = %d\n", s1, s2, s3);
```

Alıştırma 2.2 ★★☆ *İzleme*

(a) Örnek 10'daki `ebob` ile ebob(252, 105) ve ebob(144, 89) için (a, q = a / b, b, r = a % b) tablosunu doldurun; hangisi daha çok adım sürer, neden? İkinciyi Lamé sınırıyla karşılaştırın. (b) Örnek 11'deki `us_mod` ile 7^{11} mod 13 ve 2^{100} mod 1000 için tur tablosunu (b, bit, sonuç, taban) doldurun; çarpma sayısını "bit uzunluğu + 1 bitleri" formülüyle doğrulayın.

B · Karmaşıklık belirleme

Alıştırma 2.3 ★★☆ *Karmaşıklık*

Aşağıdaki b3'ün ve aynı dosyadaki b1 (adım 2), b2 (dış döngü n'den yarıya inerek, iç döngü i kez), b4 (i = 2'den, her adımda i = i * i), b5 (k < j < i üç iç içe döngü) ve b6 (iç döngü yalnızca i % 100 == 0 iken) fonksiyonlarının sınıfını Θ ile yazıp gerekçelendirin; programın yazdırdığı sayaç oranlarıyla sınayın.

İpucu: b2'de iç döngülerin toplamı geometrik seridir; b4'te i = 2, 4, 16, 256, …

C **H02_A02_karmasiklik.c · satır 32–41**

```c
long long b3(long long n)
{
    long long s = 0;
    for (long long i = 0; i * i < n; i++) {
        for (long long j = 0; j < n; j++) {
            s++;
        }
    }
    return s;
}
```

Alıştırma 2.4 ★★☆ *Karmaşıklık*

(a) Büyüme hızına göre sıralayın, aynı Θ sınıfındakileri belirtin: n log n, √n, $(\log n)^2$, 2^n, n!, n^1,5, 100n + 7, log log n, $n^2/\log n$, $\log_{10} n$, $3n^2 + 2n$, 2^{n+1}. (b) $f(n) = 2n^2 - 3n + 7$ için $\Theta(n^2)$ tanımını sağlayan c_1, c_2, n_0 bulun (Bölüm 3.3). (c) Doğru mu, yanlış mı? $f = O(n^3)$; $f = \Theta(n^3)$; $f = \Omega(n)$; $n \log n = O(n^2)$; $2^{n+1} = \Theta(2^n)$; $4^n = \Theta(2^n)$.

Alıştırma 2.5 ★★★ *Karmaşıklık*

Kart 7'deki erken çıkışlı aramanın en iyi, en kötü ve **başarılı aramalar için** ortalama karşılaştırma sayısını yazın. Aranan p olasılıkla dizideyse (dizideyse her konum eşit olasılıklı) beklenen karşılaştırma sayısı nedir? Örnek 4'ün "model" sütunuyla karşılaştırın.

C · Değişmez yazma

Alıştırma 2.6 ★★☆ *Değişmez yaz*

Örnek 11'in özyinelemeli `us_mod_oz` fonksiyonu (`c/H02_11_hizli_us.c`, satır 44–58; Ek A.11) için "dönen değer a^b mod m'dir" önermesini b üzerinde tümevarımla gerekçelendirin (temel durum b = 0). Çağrı yığını en çok kaç çerçeve derinleşir?

Alıştırma 2.7 ★★☆ *Değişmez yaz*

Bölüm 4.2'deki değişmezle gösterin: i · i > N olduğunda işaretsiz kalan her k (2 ≤ k ≤ N) asaldır. Dış döngüyü `i <= N` yapmak değişmezi bozar mı, yalnızca işi mi artırır?

D · Hata bulma

Alıştırma 2.8 ★★☆ *Hata bul*

Aşağıdaki hızlı üs fonksiyonunda iki hata var; dosyadaki küçük testler doğru sonuç verir. Hataları bulun, hangi girdilerde ortaya çıktıklarını söyleyin ve düzeltin (kâhin: Örnek 11 ya da Python `pow(a, b, m)`).

C `H02_A03_hata_bul_us.c` · satır 14–26

```c
/* a^b mod m, kare al ve çarp (ön koşul 1 <= m < 2^32) */
uint64_t us_mod(uint64_t a, uint64_t b, uint64_t m)
{
    uint64_t sonuc = 1;
    while (b > 0) {
        if (b % 2 == 1) {
            sonuc = sonuc * a % m;
        }
        a = a * a;
        b /= 2;
    }
    return sonuc;
}
```

Alıştırma 2.9 ★★☆ *Hata bul*

Tüm çiftler sayacı küçük n'de doğru, n = 100 000'de yanlış (çıktı aşağıda). Neden? Fark tam olarak ne kadar ve neyin katı? Düzeltin. (`-O2` bu döngüyü kapalı biçime indirger; `-O0` ile birkaç saniye sürer.)

C **H02_A04_hata_bul_sayac.c · satır 12–21**

```c
unsigned int tum_ciftler(unsigned int n)
{
    unsigned int sayac = 0;
    for (unsigned int i = 0; i < n; i++) {
        for (unsigned int j = i + 1; j < n; j++) {
            sayac++;
        }
    }
    return sayac;
}
```

ÇIKTI (BU MAKİNEDE)

```
$ ./H02_A04_hata_bul_sayac
n =   1000: sayaç     499500, kuram     499500
n =  10000: sayaç   49995000, kuram   49995000
n = 100000: sayaç  704982704, kuram 4999950000
```

E · Parsons problemi

Alıştırma 2.10 ★★☆ *Parsons*

Aşağıdaki Kadane fonksiyonunun satırları karıştırılmıştır (dosya: tamamlanacak; bu hâliyle derlenmez). Doğru sıraya ve girintiye koyun. Yalnızca toplamı döndüren bu sürümün Örnek 13'ten farkı nedir?

C **H02_A05_parsons_kadane.c · satır 8–19**

```c
        if (simdiki > en_iyi) {
    long long en_iyi = a[0], simdiki = a[0];
    return en_iyi;
            en_iyi = simdiki;
long long kadane(const int *a, size_t n)
        long long uzat = simdiki + a[i];
        simdiki = (uzat < a[i]) ? a[i] : uzat;
    for (size_t i = 1; i < n; i++) {
{
        }
    }
}
```

F · Kod yazma

Alıştırma 2.11 ★☆☆ *Kod yaz*

`int taban_adim(unsigned long long n, unsigned k)`: n'i her adımda k'ya bölerek kaç adımda 0'a indiğini döndürsün. k = 2, 3, 10 ve n = 1, 9, 10, 1000, $2^{63} - 1$ için sonucu $\lfloor \log_k n \rfloor + 1$ ile karşılaştıran bir `main` yazın (kayan nokta kullanmadan).

Alıştırma 2.12 ★★☆ *Kod yaz*

Bir **oran testi sürücüsü** yazın: Kart 5'in fonksiyonunu (`kavram/H02_K01_karmasiklik_kartlari.c`) n = 2^{10} … 2^{20} için `zamanlayici` modülüyle (1 ısınma + 5 tekrar, ortanca) ölçsün; her katlamada T(2n)/T(n) oranını, sonunda log–log eğimini yazsın. Oranları $2 \cdot \log(2n)/\log n$ ile karşılaştırın; sonucu kullanmayı unutmayın (Hata Avcısı 8).

Alıştırma 2.13 ★★★ *Kod yaz*

Yalnızca tek sayıları saklayan kalbur: `bilesik[k]` 2k + 1'i temsil etsin (2 ayrıca sayılır). $\pi(10^7) = 664\,579$'u doğrulayın; işaretleme sayısını ve belleği Örnek 9 ile karşılaştırın; N = 10^7 ve 10^8 için süreyi ölçün. ★★★+: işaretleri bitlere paketleyin (N/16 bayt).

G · Diller arası okuma

Alıştırma 2.14 ★★☆ *Diller arası okuma*

Bölüm 5.4'teki C `iki_us` fonksiyonu ile Ek A.12'deki Python karşılığı n = 70 için ne döndürür? C'de sonuç hangi n'den itibaren 0'dır ve neden bu davranış tanımlıdır? Python sürümünün süresi n ile nasıl büyür (Kaputun Altında 2)?

Alıştırma 2.15 ★☆☆ *Diller arası okuma*

Bölüm 4.2'deki C++ kalburu işaret dizisini satır 28'de, Python kalburu satır 19'da nasıl kuruyor? N = 10^8 için bu satırlar kaç bayt ayırır? C++'ta `std::vector<bool>` kullanılsaydı bellek ve erişim hızı nasıl değişirdi? Python'da `bytearray(n + 1)` yerine `[0] * (n + 1)` yazılsaydı?

11.2 Atölye 1 · "Karmaşıklığını söyle" kart oyunu (12 kart)

Eşli çalışın. Her kart için (a) büyüme sınıfını Θ ile (gerekiyorsa en iyi/en kötü ayrımıyla) yazın, (b) dayandığı döngü desenini gerekçelendirin (Tablo 2.3), (c) anlaşamadığınız kartları işaretleyin. 15 dakika sonra `kavram/H02_K01_karmasiklik_kartlari.c` programını çalıştırın: sayaç oranları (T(2n)/T(n)) tahminlerinizi sınar; Kart 10 ve Kart 12 (Python, `H02_K02_kart12_liste.py`) için süre ölçülür. Kartlarda fonksiyonların döngü çekirdeği gösterilir; `s` temel işlem sayacı, n girdi boyutudur (Kart 7: a dizisi ve aranan x; Kart 10: s dizgisi; Kart 11: tamsayı n; Kart 12: Python listesi a).

Tablo 2.14
"Karmaşıklığını söyle" kartları (kodlar/kavram/H02_K01_karmasiklik_kartlari.c ve H02_K02_kart12_liste.py)

Kart	Kod (C; Kart 12 Python)	Kart	Kod
Kart 1	`for (long long i = 0; i < n; i++) { s++; } for (long long j = 0; j < n; j++) { s++; }`	Kart 2	`for (long long i = 0; i < n; i++) { for (long long j = 0; j < n; j++) { s++; } }`
Kart 3	`for (long long i = 0; i < n; i++) { for (long long j = i + 1; j < n; j++) { s++; } }`	Kart 4	`for (long long i = n; i > 0; i /= 2) { s++; }`
Kart 5	`for (long long i = 0; i < n; i++) { for (long long j = 1; j < n; j *= 2) { s++; } }`	Kart 6	`for (long long i = 1; i * i <= n; i++) { s++; }`
Kart 7	`for (long long i = 0; i < n; i++) { s++; if (a[i] == x) { return s; } }`	Kart 8	`for (long long i = 0; i < n && i < 1000; i++) { s++; }`
Kart 9	`for (long long i = 1; i < n; i *= 2) { for (long long j = 0; j < i; j++) { s++; } }`	Kart 10	`for (size_t i = 0; i < strlen(s); i++) { if (s[i] == ' ') { s[i] = '_'; say++; } }`
Kart 11	`for (long long m = 0; m < (1LL << n); m++) { s++; }`	Kart 12	`for x in a: if x not in sonuc: sonuc.append(x) return sonuc`

11.3 Atölye 2 · Oran testi deneyi: üç gizemli fonksiyon

Haftanın Deneyi'ni kendi makinenizde tekrarlayın; atölyeyi Bölüm 6'yı okumadan yapmak daha öğreticidir. Kara kutular kod arşivinde kaynak olarak da vardır (`deney/` altındaki `gizemli.*` dosyaları); deney bitene kadar bu dosyaları açmayın. Öğretim üyesi isterse yalnızca `gizemli.h` başlığını ve derlenmiş hâllerini (C için `gizemli.o`, Python için `.pyc`; komutlar dosya başlıklarında) dağıtabilir.

1. Sürücüyü derleyin (Bölüm 6) ve her fonksiyon için en az yedi katlama ölçün: `./oran_deneyi 1 128 8192 5 > g1.csv`; gizemli_2 için 4096 … 1048576, gizemli_3 için 1024 … 131072.

2. Her CSV için T(2n)/T(n) oranlarını bir tabloya yazın; log–log grafiği çizin (web sayfasındaki ölçüm laboratuvarı CSV'nizi kabul eder) ve eğimi okuyun.
3. Oranı ≈ 2 olan iki fonksiyon için T(n)/n ve T(n)/(n $\log_2$ n) değerlerini hesaplayın: hangisi düz kalıyor? Kararınızı yazın.
4. Aynı ölçümü Python sürücüsüyle yapın (`python3 oran_deneyi.py 2 4096 262144 5`); eğimleri ve birim iş başına süreleri C ile ve Bölüm 6'daki tabloyla karşılaştırın.

11.4 Atölye 3 · Kadane yarışması

Örnek 13'ün süre kipiyle üç sürümü n = 10^3, 10^4 ve 10^5 için yarıştırın. İlk argüman n, ikinci argüman toplama sayısı sınırıdır: toplama sayısı bu sınırı aşan sürüm atlanır (aşağıdaki komutlarda n = 10^4).

KOMUTLAR

```
$ ./H02_13_en_buyuk_alt_dizi 10000 1e11
$ python3 H02_13_en_buyuk_alt_dizi.py 10000 2e8
```

Ölçülemeyen hücreleri oran testiyle **kestirin**: kübik sürümde n 10 kat büyürse süre ≈ 1000 kat. Bu makinedeki sonuçlar (Tablo 2.15; "(k)" kestirimdir, ölçülmedi):

Tablo 2.15
Kadane yarışması (bu makinede, tek ölçüm; C kübik kestirimi n = 4000 ölçümünden, Python kübik n = 1000'den)

Dil	Sürüm	n = 10^3	n = 10^4	n = 10^5
C -O2	kübik	76,8 ms	≈ 79,0 s (k)	≈ 21,9 saat (k)
C -O2	karesel	0,307 ms	30,7 ms	3,26 s
C -O2	Kadane	0,001 ms	0,010 ms	0,111 ms
Python	kübik	11,0 s	≈ 3,1 saat (k)	≈ 127 gün (k)
Python	karesel	50,3 ms	3,39 s	≈ 338,6 s (k)
Python	Kadane	0,156 ms	1,5 ms	8,0 ms

Görevler. (a) Kendi makinenizde tabloyu doldurun. (b) Her sürüm için 1 saniyelik bütçede çözülebilecek en büyük n'i ölçümlerinizden kestirin. (c) Python'un C'ye oranını her sürüm için hesaplayın: oran sınıfa göre değişiyor mu? (d) Üç sürümün aynı sonucu verdiğini (sağlama) denetleyin.

11.5 Atölye 4 · Asal sayma: π(10^7) = 664 579

Tablo 2.16
Asal sayma (bu makinede, tek ölçüm; Python 10^7 deneme bölmesi bölme sayısı oranıyla kestirildi)

Yöntem	N	C -O2	Python	İşlem sayısı
Kalbur	10^6	5,6 ms	201 ms	2 122 048 işaretleme
Deneme bölmesi	10^6	486 ms	6,23 s	67 740 404 bölme
Kalbur	10^7	147 ms	2,27 s	22 850 051 işaretleme
Deneme bölmesi	10^7	12,9 s	≈ 160 s (k)	1 746 210 133 bölme

C'de `./H02_09_kalbur 10000000` iki yöntemi, `./H02_09_kalbur 10000000 kalbur` yalnızca kalburu çalıştırır. Python'da 10^7 deneme bölmesi ölçülmedi; bölme sayıları oranıyla (×25,8) ≈ 160 s kestirilir. **Görevler.** (a) Tabloyu kendi makinenizde doldurun ve $\pi(10^7) = 664\,579$'u iki yöntemle doğrulayın. (b) N 10 kat büyüyünce her yöntemin süresi kaç kat büyüdü? Kuramla (N log log N ve ≈ N^1,5/log N) karşılaştırın. (c) Bellek: N = 10^8 için kalbur ne kadar bellek ister? Alıştırma 2.13'teki tek sayılı kalburla karşılaştırın.

12. Kendini Değerlendirme

- ☐ Bir döngünün temel işlemini seçip T(n)'yi kapalı biçimde yazabiliyorum (aritmetik ve geometrik seri).
- ☐ Doğrusal arama için en iyi, en kötü ve ortalama durumu ayırt edip ortalamanın hangi varsayıma dayandığını söyleyebiliyorum.
- ☐ O, Ω ve Θ'yı tanımlarıyla kullanıyor, bir örnek için c_1, c_2 ve n_0 bulabiliyorum; "O en kötü durumdur" yanılgısına düşmüyorum.
- ☐ Ardışık, iç içe, bağımlı iç içe, yarıya inen ve katlanan döngülerin sınıfını gerekçesiyle söyleyebiliyorum.
- ☐ Problem boyutundan uygulanabilir sınıfı çıkarabiliyor; oran testi ve log–log eğimle ölçümden büyüme sınıfını kestirip sabit çarpanı eğimden ayırabiliyorum.
- ☐ Deneme bölmesi, kalbur, Öklid ve hızlı üs almayı zaman ve bellek açısından karşılaştırabiliyor; Kadane'yi değişmeziyle açıklayıp kâhinle test edebiliyorum.
- ☐ C'de `strlen` ile Python'da `len` ve `x in liste` ile `x in kume` arasındaki maliyet farkını ve `uint64_t` sarmasını açıklayabiliyorum.

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · Bu haftanın konusu nerede derinleşiyor?

- Asimptotik analiz, Algoritma Analizi dersinde amortize analiz, olasılıksal analiz ve alt sınır kanıtlarıyla derinleşir.
- Kadane, H13'te dinamik programlamanın ilk örneği olarak yeniden yorumlanır.
- Hızlı üs alma H6'da böl ve fethet, kriptografi derslerinde ise RSA'nın temel işlemi olarak yeniden karşımıza çıkar.

MERAK KUTUSU · Deneme bölmesi "polinom zamanlı" mı?

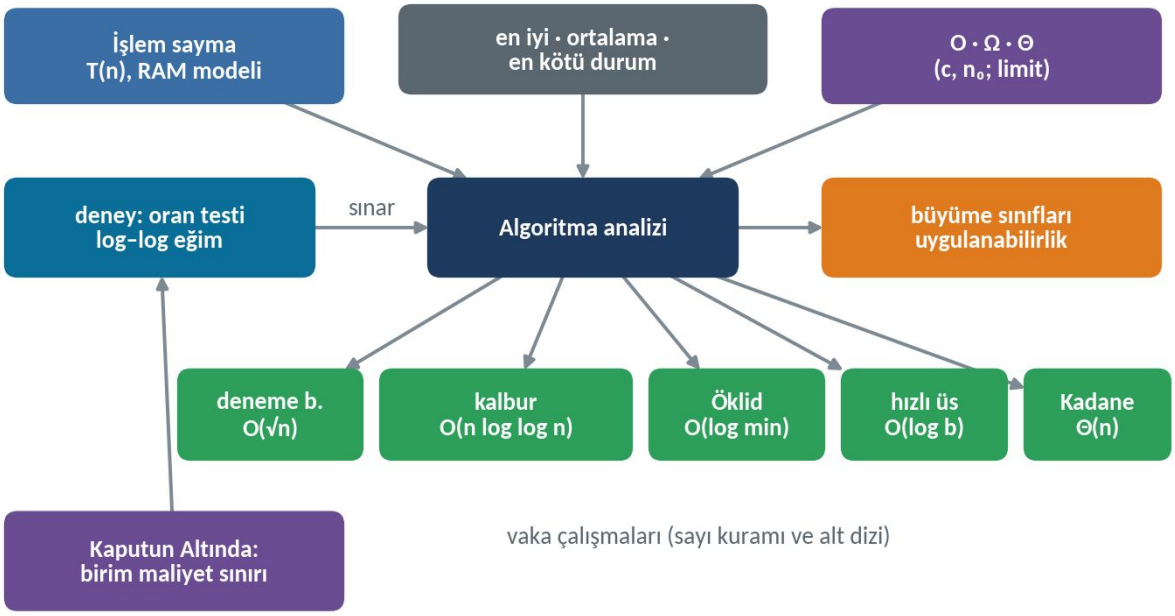
O(√n) masum görünür, ama bir sayının girdi boyutu **değeri** değil **bit sayısıdır**: b bitlik bir n için √n ≈ 2^(b/2). Yani deneme bölmesi girdi boyutuna göre **üsteldir**: 64 bitlik bir asal için ≈ 4 milyar bölme, 128 bitlik bir asal için ≈ $1{,}8 \cdot 10^{19}$. Hızlı üs alma ise bit sayısına göre doğrusal sayıda çarpma yapar; kriptografideki yüzlerce basamaklı asallar bu yüzden hızlı üs almaya dayanan olasılıksal testlerle bulunur. "Girdi boyutu nedir?" sorusu H14'te P ve NP tartışmasının kapısını açacak.

14. Özet, Kavram Haritası ve Terim Sözlüğü

- **T(n)** temel işlem sayısıdır (RAM modeli); temel işlem seçimi sabiti değiştirir, sınıfı değil. **Durum** (en iyi/ortalama/en kötü) hangi girdinin incelendiği, **gösterim** (O/Ω/Θ) bir fonksiyon hakkındaki sınırdır; ortalama durum bir olasılık varsayımına dayanır.

- **O, Ω, Θ** sabit çarpanları ve düşük dereceli terimleri atar; tanım c ve n_0 ile, pratik kullanım limit ve kurallarla yapılır. Log tabanı önemsizdir.
- **Döngü desenleri:** ardışık toplanır, iç içe çarpılır; bağımlı iç içe $n(n-1)/2$, yarıya inen $\lfloor \log_2 n \rfloor + 1$, katlanan dış döngü geometrik seri.
- **Uygulanabilirlik:** saniyede ≈ 10^8 işlemle n^2 için $n \approx 10^4$, n log n için ≈ $4{,}5 \cdot 10^6$; Büyük Soru'nun yanıtı: dil sabiti bir çarpandır, sınıf farkı girdiyle büyür.
- **Oran testi ve log–log eğim:** $T(2n)/T(n) \to 2^b$; eğim = b; küçük n'leri dışla, ortancayı kullan; n ile n log n'i düzlük testiyle ayır.
- **Sayı kuramı vakası:** deneme bölmesi O(√n), kalbur O(N log log N) zaman + O(N) bellek, Öklid O(log min(a, b)), hızlı üs O(log b); Kadane aynı problemin $O(n^3)$ ve $O(n^2)$ çözümlerini Θ(n)'e indirir.

Şekil 2.16
Hafta 2 kavram haritası



Tablo 2.17
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Türkçe	İngilizce
algoritma analizi	algorithm analysis	asimptotik gösterim	asymptotic notation
RAM modeli	random-access machine	temel işlem	basic operation
çalışma süresi T(n)	running time	kapalı biçim	closed form
en iyi / ortalama / en kötü durum	best / average / worst case	üst / alt / sıkı sınır	upper / lower / tight bound
büyüme sınıfı	order of growth	uygulanabilirlik	feasibility
sabit çarpan	constant factor	oran testi	doubling ratio test
log–log grafik, eğim	log–log plot, slope	ek bellek	auxiliary space
zaman–bellek ödünleşimi	time–memory trade-off	deneme bölmesi	trial division

Türkçe	İngilizce	Türkçe	İngilizce
Eratosthenes kalburu	sieve of Eratosthenes	hızlı üs alma	exponentiation by squaring
modüler üs	modular exponentiation	en büyük alt dizi toplamı	maximum subarray sum
kaba kuvvet	brute force	böl ve fethet	divide and conquer
döngü değişmezi	loop invariant	ön koşul	precondition
kâhin	test oracle	tanımsız davranış	undefined behavior
önbellek	cache	dal tahmini	branch prediction

15. Kaynaklar

Bentley, J. (2000). *Programming pearls* (2nd ed.). Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

t0st. (2021, February 28). *How I cut GTA Online loading times by 70%*. https://nee.lv/2021/02/28/How-I-cut-GTA-Online-loading-times-by-70/

Ek A. Dil Karşılıkları: C++ ve Python Listeleri

Bu ekte her merdiven örneğinin **çekirdek fonksiyonlarının** C++ ve Python karşılıkları yer alır; C'den farklılaşan satırlar hardal renkle vurgulanmıştır (`ornekler.json`, `farklar`). Amiral örneklerin ana fonksiyonları ve Dil Penceresi notları Bölüm 4'te üç dilde verildiği için burada kalan fonksiyonları gösteriyoruz. Dosyaların tamamı (başlık, `main`, kütüphane karşılığı notu) kod arşivindedir; satır numaraları dosyalarla aynıdır. Her örnek üç dilde aynı çıktıyı verir ("="); yalnızca Örnek 12 bilinçli olarak farklıdır ("Δ").

Tablo 2.18

Derleme ve çalıştırma: gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c [ekler] -o X && ./X · g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X && ./X · python3 X.py (C++ ortak modülleri yalnız başlıktır)

Örnek (X)	C derleme ekleri (kodlar/c/)
H02_01_dogrusal_toplam	`ortak_rastgele.c`
H02_02_tum_ciftler	—
H02_03_yariya_inen	—
H02_04_arama_durumlari	`ortak_rastgele.c`
H02_05_yinelenen_numara	`ortak_rastgele.c`
H02_06_buyume_tablosu	`-lm`
H02_07_oran_testi	`ortak_rastgele.c zamanlayici.c`
H02_08_deneme_bolmesi	—
H02_09_kalbur	`zamanlayici.c`
H02_10_oklid_lame	—
H02_11_hizli_us	—
H02_12_buyuk_tamsayi	`zamanlayici.c -lm`
H02_13_en_buyuk_alt_dizi	`ortak_rastgele.c zamanlayici.c`

A.1 Örnek 1 · H02_01_dogrusal_toplam (★, =)

C++ H02_01_dogrusal_toplam.cpp · satır 21–45

```cpp
long long dogrusal_toplam(const std::vector<int>& a, std::size_t n,
                          long long& islem)
{
    long long toplam = 0;
    islem = 0;
    for (std::size_t i = 0; i < n; i++) {
        toplam += a[i];
        islem++;                    // temel işlem: bir toplama
    }
    return toplam;
}

// Aynı döngü; RAM modelinde her ilkel adım sayılır
long long ayrintili_sayim(const std::vector<int>& a, std::size_t n,
                          long long& adim)
{
    long long toplam = 0;
    adim = 2;                       // toplam = 0 ve i = 0: 2 atama
    for (std::size_t i = 0; i < n; i++) {
        toplam += a[i];
        adim += 3;                  // i < n sınaması, toplama, i++
    }
    adim += 1;                      // döngüyü bitiren son i < n
    return toplam;
}
```

Python H02_01_dogrusal_toplam.py · satır 13–31

```python
def dogrusal_toplam(a, n):
    """a[0..n-1] toplamı ve temel işlem (toplama) sayısı."""
    toplam = 0
    islem = 0
    for i in range(n):
        toplam += a[i]
        islem += 1                  # temel işlem: bir toplama
    return toplam, islem


def ayrintili_sayim(a, n):
    """Aynı döngü; RAM modelinde her ilkel adım sayılır."""
    toplam = 0
    adim = 2                        # toplam = 0 ve i = 0: 2 atama
    for i in range(n):
        toplam += a[i]
        adim += 3                   # i < n sınaması, toplama, i++
    adim += 1                       # döngüyü bitiren son i < n
    return toplam, adim
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C++ sürümü diziyi `const std::vector<int>&` ile, sayacı referansla (`long long& islem`) alır; C'de sayaç için işaretçi (`long long *islem`) gerekir. Python fonksiyonu iki değeri demet olarak döndürür (`return toplam, islem`). Üç dilde de toplama sayısı tam n'dir: T(n) dile bağlı değildir, yalnızca süre bağlıdır.

A.2 Örnek 2 · H02_02_tum_ciftler (★, =)

C++ H02_02_tum_ciftler.cpp · satır 18–39

```cpp
long long tum_ciftler(std::size_t n)
{
    long long sayac = 0;
    for (std::size_t i = 0; i < n; i++) {
        for (std::size_t j = i + 1; j < n; j++) {   // n - 1 - i kez
            sayac++;
        }
    }
    return sayac;
}

// Tüm sıralı (i, j) çiftlerini sayar: bağımsız iç içe döngü
long long tum_sirali_ciftler(std::size_t n)
{
    long long sayac = 0;
    for (std::size_t i = 0; i < n; i++) {
        for (std::size_t j = 0; j < n; j++) {       // her i için n kez
            sayac++;
        }
    }
    return sayac;
}
```

Python H02_02_tum_ciftler.py · satır 10–25

```python
def tum_ciftler(n):
    """i < j olan (i, j) çiftlerini sayar: bağımlı iç içe döngü."""
    sayac = 0
    for i in range(n):
        for j in range(i + 1, n):       # n - 1 - i kez
            sayac += 1
    return sayac


def tum_sirali_ciftler(n):
    """Tüm sıralı (i, j) çiftlerini sayar: bağımsız iç içe döngü."""
    sayac = 0
    for i in range(n):
        for j in range(n):              # her i için n kez
            sayac += 1
    return sayac
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> Python'da `range(i + 1, n)` bağımlı iç döngüyü C'deki `for (j = i + 1; j < n; j++)` ile aynı sınırlarla kurar. Kuramsal değer `n * (n - 1) // 2` ile tamsayı olarak hesaplanır (`/` kayan nokta verirdi). Boşluk dolgusu C'de `printf("%*s", ...)`, C++'ta `std::string(6 * i, ' ')`, Python'da `" " * (6 * i)` ile yapılır.

A.3 Örnek 3 · H02_03_yariya_inen (★, =)

C++ **H02_03_yariya_inen.cpp · satır 16–35**

```cpp
int yariya_adim(unsigned long long n)
{
    int adim = 0;
    while (n > 0) {
        n /= 2;                     // tamsayı bölmesi: aşağı yuvarlar
        adim++;
    }
    return adim;
}

// Aynı döngü, üçe bölerek
int ucte_adim(unsigned long long n)
{
    int adim = 0;
    while (n > 0) {
        n /= 3;
        adim++;
    }
    return adim;
}
```

Python **H02_03_yariya_inen.py · satır 10–25**

```python
def yariya_adim(n):
    """n > 0 oldukça n'i yarıya indirir; adım sayısını döndürür."""
    adim = 0
    while n > 0:
        n //= 2                     # taban bölmesi (n >= 0'da C ile aynı)
        adim += 1
    return adim


def ucte_adim(n):
    """Aynı döngü, üçe bölerek."""
    adim = 0
    while n > 0:
        n //= 3
        adim += 1
    return adim
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C ve C++'ta `unsigned long long` en çok 2^64 − 1 değerini tutar; Python tamsayısı sınırsızdır ama aynı döngü aynı adım sayısını verir. Python'da tamsayı bölmesi `//` ile yazılır (negatif sayıda aşağı yuvarlar; burada $n \geq 0$ olduğundan C'deki `/` ile aynıdır). Kütüphane karşılığı: Python `n.bit_length()` doğrudan $\lfloor \log_2 n \rfloor + 1$ verir.

A.4 Örnek 4 · H02_04_arama_durumlari (★★, =)

C++ **H02_04_arama_durumlari.cpp · satır 21–42**

```cpp
long dogrusal_ara(const std::vector<int>& a, int x, long long& kar)
{
    kar = 0;
    for (std::size_t i = 0; i < a.size(); i++) {
        kar++;                          // temel işlem: a[i] == x
        if (a[i] == x) {
            return static_cast<long>(i);
        }
    }
    return -1;
}

// a'daki her elemanı bir kez arar: başarılı aramaların toplam sayacı
long long basarili_toplam(const std::vector<int>& a)
{
    long long toplam = 0, kar;
    for (int x : a) {
        dogrusal_ara(a, x, kar);
        toplam += kar;
    }
    return toplam;
}
```

Python **H02_04_arama_durumlari.py · satır 13–29**

```python
def dogrusal_ara(a, x):
    """a içinde x'i arar: (indis ya da -1, karşılaştırma sayısı)."""
    kar = 0
    for i in range(len(a)):
        kar += 1                        # temel işlem: a[i] == x
        if a[i] == x:
            return i, kar
    return -1, kar


def basarili_toplam(a):
    """a'daki her elemanı bir kez arar: başarılı aramaların toplam sayacı."""
    toplam = 0
    for x in a:
        _, kar = dogrusal_ara(a, x)
        toplam += kar
    return toplam
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C++ ve Python sürümleri boyutu diziden alır (`a.size()`, `len(a)`); C'de `n` ayrı parametredir. Python `dogrusal_ara` (indis, karşılaştırma) demetini döndürür; C ve C++ sayacı işaretçi/referansla verir. Kütüphane karşılıkları (`std::find`, `a.index(x)`, `x in a`) de doğrusal aramadır: en kötü durumda O(n).

A.5 Örnek 5 · H02_05_yinelenen_numara (★★, =)

C++ H02_05_yinelenen_numara.cpp · satır 21–36

```cpp
bool yinelenen_var_mi(const std::vector<long long>& no, std::size_t& ilk_i,
                      std::size_t& ilk_j, long long& kar)
{
    kar = 0;
    for (std::size_t i = 0; i + 1 < no.size(); i++) {
        for (std::size_t j = i + 1; j < no.size(); j++) {
            kar++;
            if (no[i] == no[j]) {
                ilk_i = i;
                ilk_j = j;
                return true;                    // erken çıkış
            }
        }
    }
    return false;
}
```

Python H02_05_yinelenen_numara.py · satır 12–21

```python
def yinelenen_var_mi(no):
    """İlk eşit çifti bulunca durur: (var mı, (i, j), karşılaştırma)."""
    kar = 0
    n = len(no)
    for i in range(n - 1):
        for j in range(i + 1, n):
            kar += 1
            if no[i] == no[j]:
                return True, (i, j), kar        # erken çıkış
    return False, None, kar
```

> **DİL PENCERESİ · Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> Öğrenci numarası 2 400 000 000, 32 bitlik `int`'e sığmaz (INT_MAX = 2 147 483 647): C ve C++'ta `long long`, Python'da sınırsız `int` kullanılır. Python ilk eşit çifti `(i, j)` demeti olarak, yoksa `None` döndürür; C/C++ çıktı parametreleri kullanır. Kütüphane karşılığı `len(set(no)) < len(no)` ortalama O(n)'dir (karma tablosu, H10).

A.6 Örnek 6 · H02_06_buyume_tablosu (★★, =)

C++ **H02_06_buyume_tablosu.cpp · satır 33–43**

```cpp
void iki_us_yaz(long n)
{
    double lg = static_cast<double>(n) * std::log10(2.0);
    double d = 1.0;
    if (lg < 308.0) {
        for (long i = 0; i < n; i++) {
            d *= 2.0;
        }
    }
    deger_yaz(d, lg);
}
```

Python **H02_06_buyume_tablosu.py · satır 21–29**

```python
def iki_us_yaz(n):
    """2^n: log10(2^n) = n log10(2); sığıyorsa ikiye katlayarak kesin.
    Not: 2.0 ** 10000 Python'da OverflowError verir (C'de inf olur)."""
    lg = float(n) * math.log10(2.0)
    d = 1.0
    if lg < 308.0:
        for i in range(n):
            d *= 2.0
    return deger_yaz(d, lg)
```

> DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> `%.3e`, `std::scientific << std::setprecision(3)` ve `f"{x:.3e}"` aynı biçimi (ör. `1.000e+03`) verir. Taşan değerlerde diller ayrışır: C/C++'ta `pow(2.0, 10000.0)` `inf` döndürür, Python'da `2.0 ** 10000` `OverflowError` fırlatır; bu yüzden üç sürüm de büyüklüğü log10 ile hesaplayıp `10^k` yazar ve çıktı bu makinede bayt bayt aynıdır.

A.7 Örnek 7 · H02_07_oran_testi (★★, =)

C++ **H02_07_oran_testi.cpp · satır 21–35**

```cpp
long long iki_toplam(const std::vector<int>& a, std::size_t n,
                     long long& kar)
{
    long long sayi = 0, k = 0;
    for (std::size_t i = 0; i + 1 < n; i++) {
        for (std::size_t j = i + 1; j < n; j++) {
            k++;
            if (a[i] + a[j] == 0) {
                sayi++;
            }
        }
    }
    kar = k;
    return sayi;
}
```

Python **H02_07_oran_testi.py · satır 14–23**

```python
def iki_toplam(a, n):
    """i < j ve a[i] + a[j] == 0 olan çiftler: (sayı, karşılaştırma)."""
    sayi = 0
    k = 0
    for i in range(n - 1):
        for j in range(i + 1, n):
            k += 1
            if a[i] + a[j] == 0:
                sayi += 1
    return sayi, k
```

C++ **H02_07_oran_testi.cpp · satır 84–88**

```cpp
        volatile long long havuz = 0;   // kullanılmayan sonucu -O2 siler
        double sure = olc_ortanca([&] {
            long long k;
            havuz = iki_toplam(a, static_cast<std::size_t>(n), k);
        }, static_cast<int>(tekrar)) * 1000.0;
```

Python **H02_07_oran_testi.py · satır 66–67**

```python
        sayi, kar = iki_toplam(a, n)
        sure = olc_ortanca(lambda: iki_toplam(a, n), tekrar) * 1000.0
```

> DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> Ölçülecek iş C'de `void (*)(void *)` işaretçisi ve bir yapıyla, C++'ta `[&]` lambda ile, Python'da `lambda: iki_toplam(a, n)` ile `olc_ortanca`'ya verilir. Süreler stderr'e yazıldığı için stdout üç dilde aynıdır; ölçülen süre oranları dile göre biraz değişir ama hepsi 4'e yaklaşır.

A.8 Örnek 8 · H02_08_deneme_bolmesi (★★, =)

C++ **H02_08_deneme_bolmesi.cpp · satır 39–52**

```cpp
bool asal_mi_saf(u64 n, u64& bolme)
{
    bolme = 0;
    if (n < 2) {
        return false;
    }
    for (u64 i = 2; i < n; i++) {
        bolme++;
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```

Python **H02_08_deneme_bolmesi.py · satır 28–37**

```python
def asal_mi_saf(n):
    """Saf sürüm: 2 .. n-1 arasındaki tüm adaylar."""
    bolme = 0
    if n < 2:
        return False, bolme
    for i in range(2, n):
        bolme += 1
        if n % i == 0:
            return False, bolme
    return True, bolme
```

A.9 Örnek 9 · H02_09_kalbur (★★, =)

C++ **H02_09_kalbur.cpp · satır 64–76**

```cpp
long deneme_say(std::uint64_t n, long long& toplam_bolme)
{
    long asal = 0;
    std::uint64_t b;
    toplam_bolme = 0;
    for (std::uint64_t m = 2; m <= n; m++) {
        if (asal_mi(m, b)) {
            asal++;
        }
        toplam_bolme += static_cast<long long>(b);
    }
    return asal;
}
```

Python **H02_09_kalbur.py · satır 49–58**

```python
def deneme_say(n):
    """2..n arasındaki asalları deneme bölmesiyle sayar."""
    asal = 0
    toplam_bolme = 0
    for m in range(2, n + 1):
        a, b = asal_mi(m)
        if a:
            asal += 1
        toplam_bolme += b
    return asal, toplam_bolme
```

A.10 Örnek 10 · H02_10_oklid_lame (★★★, =)

C++ H02_10_oklid_lame.cpp · satır 19–53

```cpp
std::uint64_t ebob(std::uint64_t a, std::uint64_t b, int& adim)
{
    adim = 0;
    while (b != 0) {
        std::uint64_t r = a % b;
        a = b;
        b = r;
        adim++;
    }
    return a;
}

// Her adımı a = q * b + r biçiminde yazar
void ebob_izle(std::uint64_t a, std::uint64_t b)
{
    while (b != 0) {
        std::cout << "  " << a << " = " << a / b << " * " << b << " + "
                  << a % b << '\n';
        std::uint64_t r = a % b;
        a = b;
        b = r;
    }
    std::cout << "  ebob = " << a << '\n';
}

// Ondalık basamak sayısı
int basamak(std::uint64_t x)
{
    int d = 1;
    while (x >= 10) {
        x /= 10;
        d++;
    }
    return d;
}
```

Python **H02_10_oklid_lame.py · satır 12–35**

```python
def ebob(a, b):
    """ebob(a, b), a >= b >= 0: (ebob, kalan (mod) işlemi sayısı)."""
    adim = 0
    while b != 0:
        a, b = b, a % b
        adim += 1
    return a, adim


def ebob_izle(a, b):
    """Her adımı a = q * b + r biçiminde yazar."""
    while b != 0:
        print(f"  {a} = {a // b} * {b} + {a % b}")
        a, b = b, a % b
    print(f"  ebob = {a}")


def basamak(x):
    """Ondalık basamak sayısı."""
    d = 1
    while x >= 10:
        x //= 10
        d += 1
    return d
```

C++ **H02_10_oklid_lame.cpp · satır 59–71**

```cpp
    std::vector<std::uint64_t> fib(94);
    int adim;

    std::cout << "İzleme: ebob(1071, 462)\n";
    ebob_izle(1071, 462);
    std::cout << "\nİzleme: ebob(89, 55) — ardışık Fibonacci sayıları\n";
    ebob_izle(89, 55);

    fib[1] = 1;
    fib[2] = 1;
    for (int k = 3; k <= 93; k++) {
        fib[k] = fib[k - 1] + fib[k - 2];   // F(93) < 2^64
    }
```

Python **H02_10_oklid_lame.py · satır 48–51**

```python
    fib[1] = 1
    fib[2] = 1
    for k in range(3, 94):
        fib[k] = fib[k - 1] + fib[k - 2]    # Python'da taşma yok
```

DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**

Python `a, b = b, a % b` çoklu atamasıyla geçici değişkene gerek duymaz; C/C++'ta `r = a % b; a = b; b = r;` yazılır. Fibonacci tablosu C/C++'ta `uint64_t` ile F(93)'te durur (F(94) > 2^64); Python'da sınır yoktur. Kütüphane karşılığı: C++17 `std::gcd`, Python `math.gcd`.

A.11 Örnek 11 · H02_11_hizli_us (★★★, =)

C++ **H02_11_hizli_us.cpp · satır 47–60**

```cpp
u64 us_mod_oz(u64 a, u64 b, u64 m, int& carpma)
{
    if (b == 0) {
        return 1 % m;
    }
    u64 yarim = us_mod_oz(a, b / 2, m, carpma);
    u64 sonuc = yarim * yarim % m;
    carpma++;
    if (b % 2 == 1) {
        sonuc = sonuc * (a % m) % m;
        carpma++;
    }
    return sonuc;
}
```

Python **H02_11_hizli_us.py · satır 35–45**

```python
def us_mod_oz(a, b, m):
    """a^b mod m, özyinelemeli: a^b = (a^(b/2))² · (b tekse a)."""
    if b == 0:
        return 1 % m, 0
    yarim, carpma = us_mod_oz(a, b // 2, m)
    sonuc = yarim * yarim % m
    carpma += 1
    if b % 2 == 1:
        sonuc = sonuc * (a % m) % m
        carpma += 1
    return sonuc, carpma
```

A.12 Örnek 12 · H02_12_buyuk_tamsayi (★★★, Δ)

C++ H02_12_buyuk_tamsayi.cpp · satır 21–39

```cpp
std::uint64_t iki_us(int n)
{
    std::uint64_t x = 1;
    for (int i = 0; i < n; i++) {
        x = x * 2;
    }
    return x;
}

// x'in ikilik basamak (bit) sayısı; x = 0 için 0
int bit_uzunlugu(std::uint64_t x)
{
    int b = 0;
    while (x > 0) {
        x /= 2;
        b++;
    }
    return b;
}
```

Python H02_12_buyuk_tamsayi.py · satır 17–27

```python
def iki_us(n):
    x = 1
    for i in range(n):
        x = x * 2
    return x


def bit_uzunlugu(x):
    """x'in ikilik basamak (bit) sayısı; x = 0 için 0. Yerleşik yöntem:
    C'deki yarıya bölme döngüsü 10^5 bitlik sayıda O(k²) iş yapardı."""
    return x.bit_length()
```

> DİL PENCERESİ · **Farklılaşan satırlar (hardal) ve Dil Penceresi**
>
> C ve C++'ta `uint64_t` işaretsiz olduğundan taşma tanımlıdır: değer 2^64 modunda sarar ve n ≥ 64 için 2^n sonucu 0 olur. Python `int`'i büyüyerek kesin değeri tutar (2^100000 yaklaşık 13 KB), ama bedeli süredir: k bitlik sayıyı ikiye katlamak O(k) olduğundan 2^n'in hesabı O(n²) bit işlemidir. `pow(3.0, 39.0)` üç dilde de 53 bitlik `double`'a yuvarlanır ve tam değerden 11 eksik çıkar.

ÇIKTI · C (C++ AYNI)

```
$ ./H02_12_buyuk_tamsayi
2^n, ikiye katlayarak (uint64_t)
     n  bit_uzunluğu  son_6_basamak  mod_1000000007
     1             2         000002               2
    10            11         001024            1024
    63            64         775808       291172004
    64             0         000000               0
    65             0         000000               0
   100             0         000000               0
  1000             0         000000               0
 10000             0         000000               0
100000             0         000000               0

3^39 tamsayı çarpımıyla : 4052555153018976267
3^39 pow() ile (double) : 4052555153018976256
fark                    : 11
```

ÇIKTI · PYTHON

```
$ python3 H02_12_buyuk_tamsayi.py
2^n, ikiye katlayarak (Python int)
     n  bit_uzunluğu  son_6_basamak  mod_1000000007
     1             2         000002               2
    10            11         001024            1024
    63            64         775808       291172004
    64            65         551616       582344008
    65            66         103232       164688009
   100           101         205376       976371285
  1000          1001         069376       688423210
 10000         10001         709376       905611805
100000        100001         109376       607723520

3^39 tamsayı çarpımıyla : 4052555153018976267
3^39 pow() ile (double) : 4052555153018976256
fark                    : 11
```

A.13 Örnek 13 · H02_13_en_buyuk_alt_dizi (★★★, =)

C++ H02_13_en_buyuk_alt_dizi.cpp · satır 52–68

```cpp
Sonuc karesel(const std::vector<int>& a, std::size_t n)
{
    Sonuc s = {a[0], 0, 0, 0};
    for (std::size_t i = 0; i < n; i++) {
        long long toplam = 0;
        for (std::size_t j = i; j < n; j++) {
            toplam += a[j];
            s.topl++;
            if (toplam > s.toplam) {
                s.toplam = toplam;
                s.bas = i;
                s.son = j;
            }
        }
    }
    return s;
}
```

Python H02_13_en_buyuk_alt_dizi.py · satır 34–44

```python
def karesel(a, n):
    """O(n²): a[i..j] toplamı, a[i..j-1] toplamına a[j] eklenerek."""
    en_iyi, bas, son, topl = a[0], 0, 0, 0
    for i in range(n):
        toplam = 0
        for j in range(i, n):
            toplam += a[j]
            topl += 1
            if toplam > en_iyi:
                en_iyi, bas, son = toplam, i, j
    return en_iyi, bas, son, topl
```